DATA MINING COURSE READINGS

Chapter 03: Classification · Decision Trees & Model Evaluati

Curated Textbook Readings from Tan et al. · Matching Lecture: IE500_DM_03_Classification_Part1.pdf

REFERENCE TEXTBOOK

Introduction to Data Mining (2nd Edition, Global Edition)

Authors: Pang-Ning Tan, Michael Steinbach, Anuj Karpatne, Vipin Kumar | Pearson, 2019

Curated by: Ehsan Shahbazi | Shahid Beheshti University (SBU)

INCLUDED CHAPTER SECTIONS & TOPIC SCOPE

- **Sections 3.1·3.3: Decision Tree Classifiers** — Book p. 135·167
  Hunt's algorithm, attribute test conditions, Gini index, Entropy, Gain ratio, tree induction
- **Section 3.4: Model Overfitting** — Book p. 168·176
  Reasons for overfitting, noise, Occam's razor, pre-pruning and post-pruning methods
- **Section 6.10: Ensemble Methods** — Book p. 499·515
  Ensemble rationale, Bagging, Boosting (AdaBoost), Random Forests and out-of-bag error
- **Sections 6.11.2·6.11.4: Performance Metrics & ROC** — Book p. 521·533
  Confusion matrix, precision, recall, F-measure, cost-sensitive evaluation, ROC & AUC

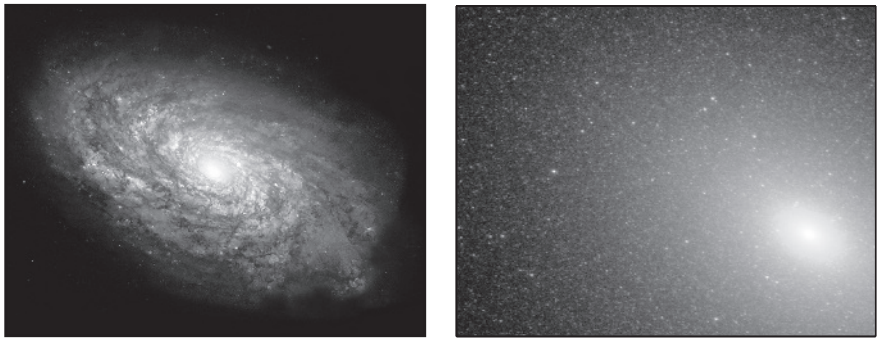
STUDENT STUDY GUIDE & SCOPE

Focuses on supervised learning with Decision Trees, impurity measures (Gini, Entropy), combating overfitting, ensemble methods (Bagging, Boosting, Random Forests), and evaluation metrics (Confusion Matrix, Precision/Recall, ROC).

3

Classification: Basic Concepts and Techniques

Humans have an innate ability to classify things into categories, e.g., mundane tasks such as filtering spam email messages or more specialized tasks such as recognizing celestial objects in telescope images (see Figure 3.1). While manual classification often suffices for small and simple data sets with only a few attributes, larger and more complex data sets require an automated solution.



(a) A spiral galaxy. (b) An elliptical galaxy.

Figure 3.1. Classification of galaxies from telescope images taken from the NASA website.



Figure 3.2. A schematic illustration of a classification task.

This chapter introduces the basic concepts of classification and describes some of its key issues such as model overfitting, model selection, and model evaluation. While these topics are illustrated using a classification technique known as decision tree induction, most of the discussion in this chapter is also applicable to other classification techniques, many of which are covered in Chapter 6.

3.1 Basic Concepts

Figure 3.2 illustrates the general idea behind classification. The data for a classification task consists of a collection of instances (records). Each such instance is characterized by the tuple $(\mathbf{x}, y)$, where $\mathbf{x}$ is the set of attribute values that describe the instance and y is the class label of the instance. The attribute set $\mathbf{x}$ can contain attributes of any type, while the class label y must be categorical.

A **classification model** is an abstract representation of the relationship between the attribute set and the class label. As will be seen in the next two chapters, the model can be represented in many ways, e.g., as a tree, a probability table, or simply, a vector of real-valued parameters. More formally, we can express it mathematically as a target function f that takes as input the attribute set $\mathbf{x}$ and produces an output corresponding to the predicted class label. The model is said to classify an instance $(\mathbf{x}, y)$ correctly if $f(\mathbf{x}) = y$.

Table 3.1. Examples of classification tasks.

Task	Attribute set	Class label
Spam filtering	Features extracted from email message header and content	spam or non-spam
Tumor identification	Features extracted from magnetic resonance imaging (MRI) scans	malignant or benign
Galaxy classification	Features extracted from telescope images	elliptical, spiral, or irregular-shaped

Table 3.2. A sample data for the vertebrate classification problem.

Vertebrate Name	Body Temperature	Skin Cover	Gives Birth	Aquatic Creature	Aerial Creature	Has Legs	Hibernates	Class Label
human	warm-blooded	hair	yes	no	no	yes	no	mammal
python	cold-blooded	scales	no	no	no	no	yes	reptile
salmon	cold-blooded	scales	no	yes	no	no	no	fish
whale	warm-blooded	hair	yes	yes	no	no	no	mammal
frog	cold-blooded	none	no	semi	no	yes	yes	amphibian
komodo dragon	cold-blooded	scales	no	no	no	yes	no	reptile
bat	warm-blooded	hair	yes	no	yes	yes	yes	mammal
pigeon	warm-blooded	feathers	no	no	yes	yes	no	bird
cat	warm-blooded	fur	yes	no	no	yes	no	mammal
leopard shark	cold-blooded	scales	yes	yes	no	no	no	fish
turtle	cold-blooded	scales	no	semi	no	yes	no	reptile
penguin	warm-blooded	feathers	no	semi	no	yes	no	bird
porcupine	warm-blooded	quills	yes	no	no	yes	yes	mammal
eel	cold-blooded	scales	no	yes	no	no	no	fish
salamander	cold-blooded	none	no	semi	no	yes	yes	amphibian

Table 3.1 shows examples of attribute sets and class labels for various classification tasks. Spam filtering and tumor identification are examples of binary classification problems, in which each data instance can be categorized into one of two classes. If the number of classes is larger than 2, as in the galaxy classification example, then it is called a multiclass classification problem.

We illustrate the basic concepts of classification in this chapter with the following two examples.

Example 3.1. [Vertebrate Classification] Table 3.2 shows a sample data set for classifying vertebrates into mammals, reptiles, birds, fishes, and amphibians. The attribute set includes characteristics of the vertebrate such as its body temperature, skin cover, and ability to fly. The data set can also be used for a binary classification task such as mammal classification, by grouping the reptiles, birds, fishes, and amphibians into a single category called non-mammals. ■

Example 3.2. [Loan Borrower Classification] Consider the problem of predicting whether a loan borrower will repay the loan or default on the loan payments. The data set used to build the classification model is shown in Table 3.3. The attribute set includes personal information of the borrower such as marital status and annual income, while the class label indicates whether the borrower had defaulted on the loan payments. ■

Table 3.3. A sample data for the loan borrower classification problem.

ID	Home Owner	Marital Status	Annual Income	Defaulted?
1	Yes	Single	125000	No
2	No	Married	100000	No
3	No	Single	70000	No
4	Yes	Married	120000	No
5	No	Divorced	95000	Yes
6	No	Single	60000	No
7	Yes	Divorced	220000	No
8	No	Single	85000	Yes
9	No	Married	75000	No
10	No	Single	90000	Yes

A classification model serves two important roles in data mining. First, it is used as a **predictive model** to classify previously unlabeled instances. A good classification model must provide accurate predictions with a fast response time. Second, it serves as a **descriptive model** to identify the characteristics that distinguish instances from different classes. This is particularly useful for critical applications, such as medical diagnosis, where it is insufficient to have a model that makes a prediction without justifying how it reaches such a decision.

For example, a classification model induced from the vertebrate data set shown in Table 3.2 can be used to predict the class label of the following vertebrate:

Vertebrate Name	Body Temperature	Skin Cover	Gives Birth	Aquatic Creature	Aerial Creature	Has Legs	Hiber-nates	Class Label
gila monster	cold-blooded	scales	no	no	no	yes	yes	?

In addition, it can be used as a descriptive model to help determine characteristics that define a vertebrate as a mammal, a reptile, a bird, a fish, or an amphibian. For example, the model may identify mammals as warm-blooded vertebrates that give birth to their young.

There are several points worth noting regarding the previous example. First, although all the attributes shown in Table 3.2 are qualitative, there are no restrictions on the type of attributes that can be used as predictor variables. The class label, on the other hand, must be of nominal type. This distinguishes classification from other predictive modeling tasks such as regression, where the predicted value is often quantitative. More information about regression can be found in Appendix D.

Another point worth noting is that not all attributes may be relevant to the classification task. For example, the average length or weight of a

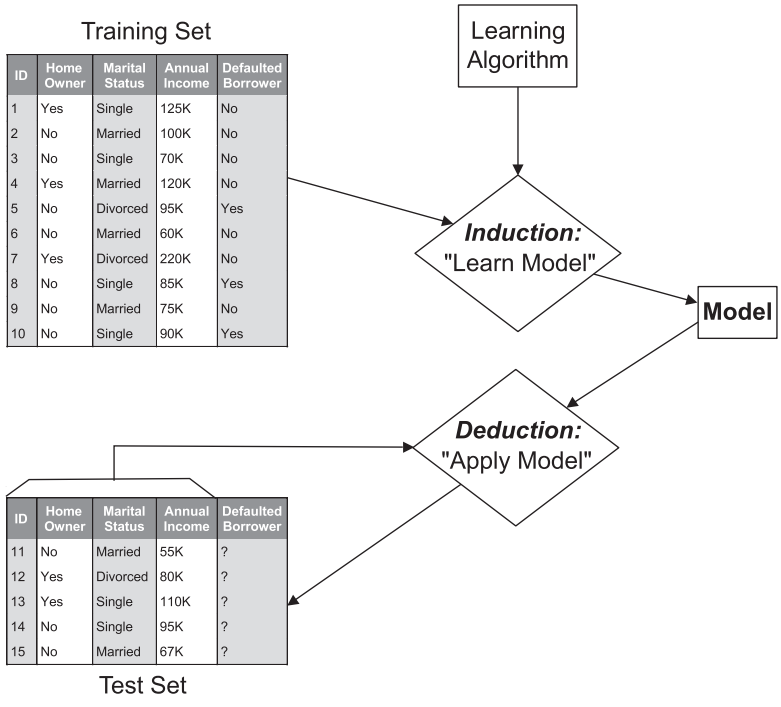
vertebrate may not be useful for classifying mammals, as these attributes can show same value for both mammals and non-mammals. Such an attribute is typically discarded during preprocessing. The remaining attributes might not be able to distinguish the classes by themselves, and thus, must be used in concert with other attributes. For instance, the `Body Temperature` attribute is insufficient to distinguish mammals from other vertebrates. When it is used together with `Gives Birth`, the classification of mammals improves significantly. However, when additional attributes, such as `Skin Cover` are included, the model becomes overly specific and no longer covers all mammals. Finding the optimal combination of attributes that best discriminates instances from different classes is the key challenge in building classification models.

3.2 General Framework for Classification

Classification is the task of assigning labels to unlabeled data instances and a **classifier** is used to perform such a task. A classifier is typically described in terms of a model as illustrated in the previous section. The model is created using a given a set of instances, known as the **training set**, which contains attribute values as well as class labels for each instance. The systematic approach for learning a classification model given a training set is known as a **learning algorithm**. The process of using a learning algorithm to build a classification model from the training data is known as **induction**. This process is also often described as "learning a model" or "building a model." This process of applying a classification model on unseen test instances to predict their class labels is known as **deduction**. Thus, the process of classification involves two steps: applying a learning algorithm to training data to learn a model, and then applying the model to assign labels to unlabeled instances. Figure 3.3 illustrates the general framework for classification.

A **classification technique** refers to a general approach to classification, e.g., the decision tree technique that we will study in this chapter. This classification technique like most others, consists of a family of related models and a number of algorithms for learning these models. In Chapter 6, we will study additional classification techniques, including neural networks and support vector machines.

A couple notes on terminology. First, the terms "classifier" and "model" are often taken to be synonymous. If a classification technique builds a single, global model, then this is fine. However, while every model defines a classifier, not every classifier is defined by a single model. Some classifiers, such as k-nearest neighbor classifiers, do not build an explicit model (Section 6.3), while



ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
11	No	Married	55K	?
12	Yes	Divorced	80K	?
13	Yes	Single	110K	?
14	No	Single	95K	?
15	No	Married	67K	?

Figure 3.3. General framework for building a classification model.

other classifiers, such as ensemble classifiers, combine the output of a collection of models (Section 6.10). Second, the term "classifier" is often used in a more general sense to refer to a classification technique. Thus, for example, "decision tree classifier" can refer to the decision tree classification technique or a specific classifier built using that technique. Fortunately, the meaning of "classifier" is usually clear from the context.

In the general framework shown in Figure 3.3, the induction and deduction steps should be performed separately. In fact, as will be discussed later in Section 3.6, the training and test sets should be independent of each other to ensure that the induced model can accurately predict the class labels of instances it has never encountered before. Models that deliver such predictive insights are said to have good **generalization performance**. The performance of a model (classifier) can be evaluated by comparing the predicted labels against the true labels of instances. This information can be summarized in a table called a **confusion matrix**. Table 3.4 depicts the confusion matrix for a binary classification problem. Each entry f_{ij} denotes the number of instances from class i predicted to be of class j. For example, f_{01} is the

Table 3.4. Confusion matrix for a binary classification problem.

		Predicted Class	
		Class = 1	Class = 0
Actual Class	Class = 1	f_{11}	f_{10}
	Class = 0	f_{01}	f_{00}

number of instances from class 0 incorrectly predicted as class 1. The number of correct predictions made by the model is $(f_{11} + f_{00})$ and the number of incorrect predictions is $(f_{10} + f_{01})$.

Although a confusion matrix provides the information needed to determine how well a classification model performs, summarizing this information into a single number makes it more convenient to compare the relative performance of different models. This can be done using an **evaluation metric** such as **accuracy**, which is computed in the following way:

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total number of predictions}}. \tag{3.1}$$

For binary classification problems, the accuracy of a model is given by

$$\text{Accuracy} = \frac{f_{11} + f_{00}}{f_{11} + f_{10} + f_{01} + f_{00}}. \tag{3.2}$$

Error rate is another related metric, which is defined as follows for binary classification problems:

$$\text{Error rate} = \frac{\text{Number of wrong predictions}}{\text{Total number of predictions}} = \frac{f_{10} + f_{01}}{f_{11} + f_{10} + f_{01} + f_{00}}. \tag{3.3}$$

The learning algorithms of most classification techniques are designed to learn models that attain the highest accuracy, or equivalently, the lowest error rate when applied to the test set. We will revisit the topic of model evaluation in Section 3.6.

3.3 Decision Tree Classifier

This section introduces a simple classification technique known as the **decision tree** classifier. To illustrate how a decision tree works, consider the classification problem of distinguishing mammals from non-mammals using

the vertebrate data set shown in Table 3.2. Suppose a new species is discovered by scientists. How can we tell whether it is a mammal or a non-mammal? One approach is to pose a series of questions about the characteristics of the species. The first question we may ask is whether the species is cold- or warm-blooded. If it is cold-blooded, then it is definitely not a mammal. Otherwise, it is either a bird or a mammal. In the latter case, we need to ask a follow-up question: Do the females of the species give birth to their young? Those that do give birth are definitely mammals, while those that do not are likely to be non-mammals (with the exception of egg-laying mammals such as the platypus and spiny anteater).

The previous example illustrates how we can solve a classification problem by asking a series of carefully crafted questions about the attributes of the test instance. Each time we receive an answer, we could ask a follow-up question until we can conclusively decide on its class label. The series of questions and their possible answers can be organized into a hierarchical structure called a decision tree. Figure 3.4 shows an example of the decision tree for the mammal classification problem. The tree has three types of nodes:

- A **root node**, with no incoming links and zero or more outgoing links.
- **Internal nodes**, each of which has exactly one incoming link and two or more outgoing links.
- **Leaf** or **terminal** nodes, each of which has exactly one incoming link and no outgoing links.

Every leaf node in the decision tree is associated with a class label. The **non-terminal** nodes, which include the root and internal nodes, contain **attribute test conditions** that are typically defined using a single attribute. Each possible outcome of the attribute test condition is associated with exactly one child of this node. For example, the root node of the tree shown in Figure 3.4 uses the attribute `Body Temperature` to define an attribute test condition that has two outcomes, warm and cold, resulting in two child nodes.

Given a decision tree, classifying a test instance is straightforward. Starting from the root node, we apply its attribute test condition and follow the appropriate branch based on the outcome of the test. This will lead us either to another internal node, for which a new attribute test condition is applied, or to a leaf node. Once a leaf node is reached, we assign the class label associated with the node to the test instance. As an illustration, Figure 3.5 traces the path used to predict the class label of a flamingo. The path terminates at a leaf node labeled as `Non-mammals`.

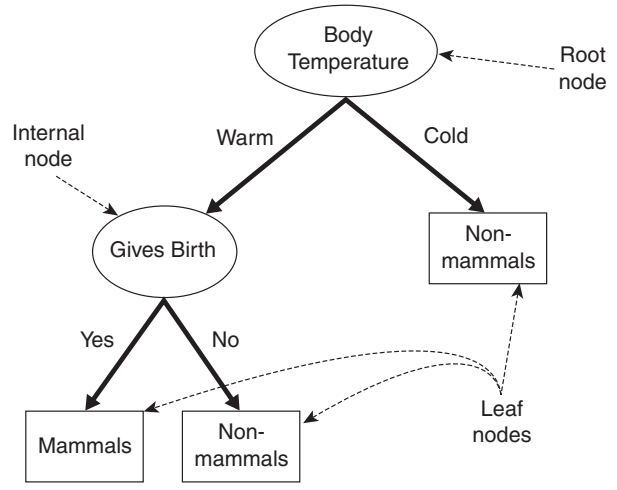


Figure 3.4. A decision tree for the mammal classification problem.

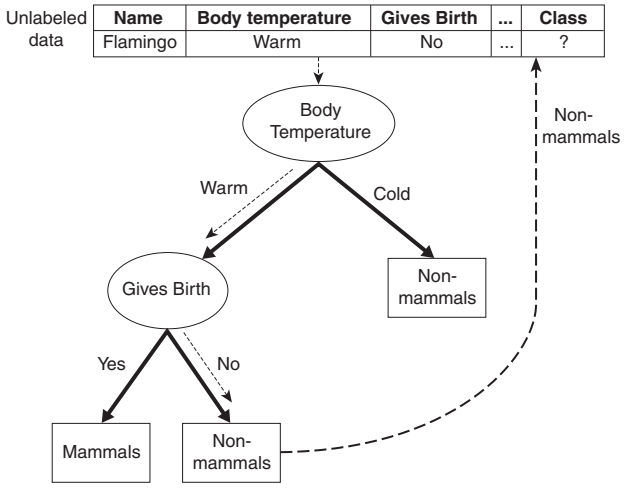


Figure 3.5. Classifying an unlabeled vertebrate. The dashed lines represent the outcomes of applying various attribute test conditions on the unlabeled vertebrate. The vertebrate is eventually assigned to the `Non-mammals` class.

3.3.1 A Basic Algorithm to Build a Decision Tree

Many possible decision trees that can be constructed from a particular data set. While some trees are better than others, finding an optimal one is computationally expensive due to the exponential size of the search space. Efficient algorithms have been developed to induce a reasonably accurate, albeit

suboptimal, decision tree in a reasonable amount of time. These algorithms usually employ a greedy strategy to grow the decision tree in a top-down fashion by making a series of locally optimal decisions about which attribute to use when partitioning the training data. One of the earliest method is **Hunt's algorithm**, which is the basis for many current implementations of decision tree classifiers, including ID3, C4.5, and CART. This subsection presents Hunt's algorithm and describes some of the design issues that must be considered when building a decision tree.

Hunt's Algorithm

In Hunt's algorithm, a decision tree is grown in a recursive fashion. The tree initially contains a single root node that is associated with all the training instances. If a node is associated with instances from more than one class, it is expanded using an attribute test condition that is determined using a **splitting criterion**. A child leaf node is created for each outcome of the attribute test condition and the instances associated with the parent node are distributed to the children based on the test outcomes. This node expansion step can then be recursively applied to each child node, as long as it has labels of more than one class. If all the instances associated with a leaf node have identical class labels, then the node is not expanded any further. Each leaf node is assigned a class label that occurs most frequently in the training instances associated with the node.

To illustrate how the algorithm works, consider the training set shown in Table 3.3 for the loan borrower classification problem. Suppose we apply Hunt's algorithm to fit the training data. The tree initially contains only a single leaf node as shown in Figure 3.6(a). This node is labeled as `Defaulted = No`, since the majority of the borrowers did not default on their loan payments. The training error of this tree is 30% as three out of the ten training instances have the class label `Defaulted = Yes`. The leaf node can therefore be further expanded because it contains training instances from more than one class.

Let `Home Owner` be the attribute chosen to split the training instances. The justification for choosing this attribute as the attribute test condition will be discussed later. The resulting binary split on the `Home Owner` attribute is shown in Figure 3.6(b). All the training instances for which `Home Owner = Yes` are propagated to the left child of the root node and the rest are propagated to the right child. Hunt's algorithm is then recursively applied to each child. The left child becomes a leaf node labeled `Defaulted = No`, since all instances associated with this node have identical class label `Defaulted = No`. The right child has instances from each class label. Hence, we split it

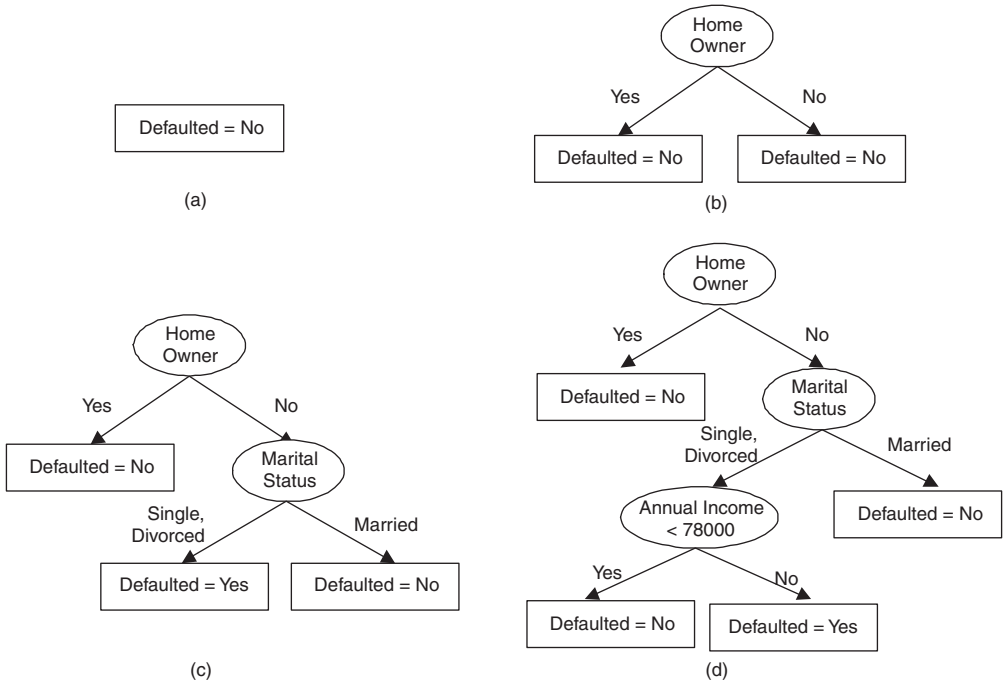


Figure 3.6. Hunt's algorithm for building decision trees.

further. The resulting subtrees after recursively expanding the right child are shown in Figures 3.6(c) and (d).

Hunt's algorithm, as described above, makes some simplifying assumptions that are often not true in practice. In the following, we describe these assumptions and briefly discuss some of the possible ways for handling them.

1. Some of the child nodes created in Hunt's algorithm can be empty if none of the training instances have the particular attribute values. One way to handle this is by declaring each of them as a leaf node with a class label that occurs most frequently among the training instances associated with their parent nodes.

2. If all training instances associated with a node have identical attribute values but different class labels, it is not possible to expand this node any further. One way to handle this case is to declare it a leaf node and assign it the class label that occurs most frequently in the training instances associated with this node.

Design Issues of Decision Tree Induction

Hunt's algorithm is a generic procedure for growing decision trees in a greedy fashion. To implement the algorithm, there are two key design issues that must be addressed.

1. **What is the splitting criterion?** At each recursive step, an attribute must be selected to partition the training instances associated with a node into smaller subsets associated with its child nodes. The splitting criterion determines which attribute is chosen as the test condition and how the training instances should be distributed to the child nodes. This will be discussed in Sections 3.3.2 and 3.3.3.

2. **What is the stopping criterion?** The basic algorithm stops expanding a node only when all the training instances associated with the node have the same class labels or have identical attribute values. Although these conditions are sufficient, there are reasons to stop expanding a node much earlier even if the leaf node contains training instances from more than one class. This process is called early termination and the condition used to determine when a node should be stopped from expanding is called a stopping criterion. The advantages of early termination are discussed in Section 3.4.

3.3.2 Methods for Expressing Attribute Test Conditions

Decision tree induction algorithms must provide a method for expressing an attribute test condition and its corresponding outcomes for different attribute types.

Binary Attributes The test condition for a binary attribute generates two potential outcomes, as shown in Figure 3.7.

Nominal Attributes Since a nominal attribute can have many values, its attribute test condition can be expressed in two ways, as a multiway split or a binary split as shown in Figure 3.8. For a multiway split (Figure 3.8(a)), the number of outcomes depends on the number of distinct values for the corresponding attribute. For example, if an attribute such as marital status has three distinct values—single, married, or divorced—its test condition will produce a three-way split. It is also possible to create a binary split by partitioning all values taken by the nominal attribute into two groups. For example, some decision tree algorithms, such as CART, produce only binary

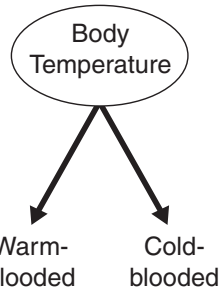


Figure 3.7. Attribute test condition for a binary attribute.

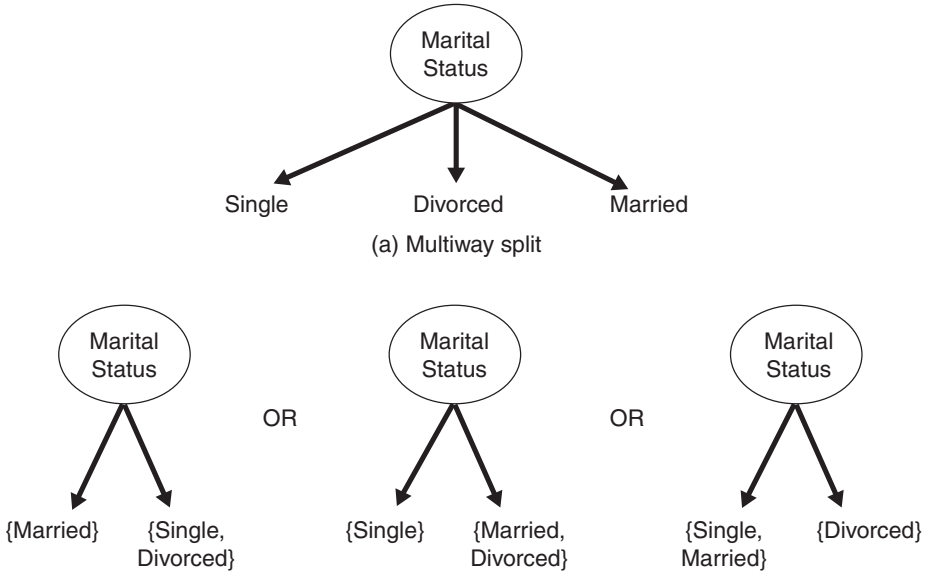


Figure 3.8. Attribute test conditions for nominal attributes.

splits by considering all $2^{k-1} - 1$ ways of creating a binary partition of k attribute values. Figure 3.8(b) illustrates three different ways of grouping the attribute values for marital status into two subsets.

Ordinal Attributes Ordinal attributes can also produce binary or multiway splits. Ordinal attribute values can be grouped as long as the grouping

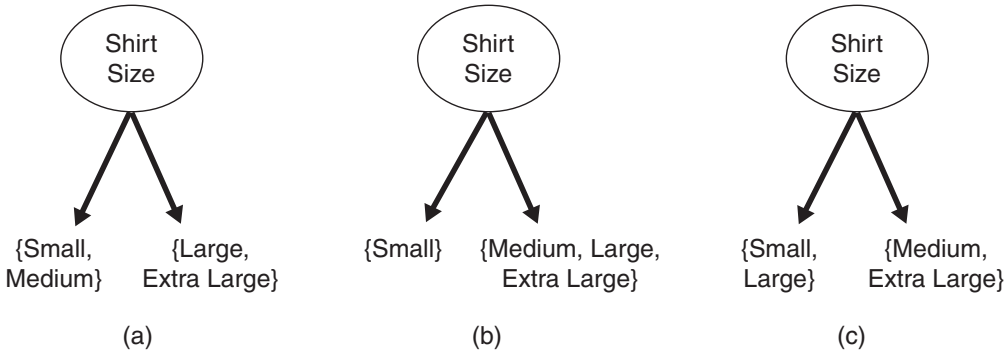


Figure 3.9. Different ways of grouping ordinal attribute values.

does not violate the order property of the attribute values. Figure 3.9 illustrates various ways of splitting training records based on the `Shirt Size` attribute. The groupings shown in Figures 3.9(a) and (b) preserve the order among the attribute values, whereas the grouping shown in Figure 3.9(c) violates this property because it combines the attribute values `Small` and `Large` into the same partition while `Medium` and `Extra Large` are combined into another partition.

Continuous Attributes For continuous attributes, the attribute test condition can be expressed as a comparison test (e.g., $A < v$) producing a binary split, or as a range query of the form $v_i \leq A < v_{i+1}$, for $i = 1, \ldots, k$, producing a multiway split. The difference between these approaches is shown in Figure 3.10. For the binary split, any possible value v between the minimum and maximum attribute values in the training data can be used for constructing the comparison test $A < v$. However, it is sufficient to only consider distinct attribute values in the training set as candidate split positions. For the multiway split, any possible collection of attribute value ranges can be used, as long as they are mutually exclusive and cover the entire range of attribute values between the minimum and maximum values observed in the training set. One approach for constructing multiway splits is to apply the discretization strategies described in Section 2.3.6 on page 83. After discretization, a new ordinal value is assigned to each discretized interval, and the attribute test condition is then defined using this newly constructed ordinal attribute.

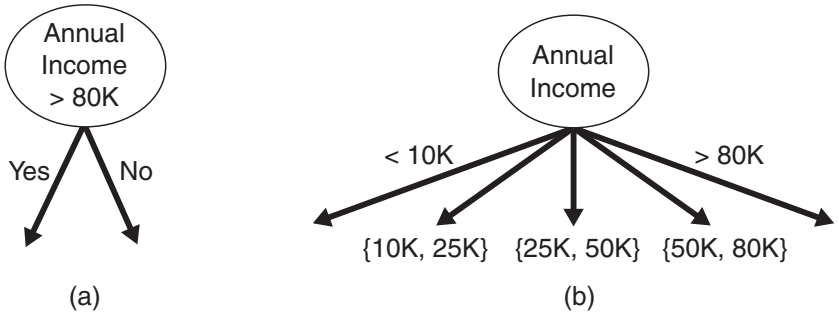


Figure 3.10. Test condition for continuous attributes.

3.3.3 Measures for Selecting an Attribute Test Condition

There are many measures that can be used to determine the goodness of an attribute test condition. These measures try to give preference to attribute test conditions that partition the training instances into *purer* subsets in the child nodes, which mostly have the same class labels. Having purer nodes is useful since a node that has all of its training instances from the same class does not need to be expanded further. In contrast, an impure node containing training instances from multiple classes is likely to require several levels of node expansions, thereby increasing the depth of the tree considerably. Larger trees are less desirable as they are more susceptible to model overfitting, a condition that may degrade the classification performance on unseen instances, as will be discussed in Section 3.4. They are also difficult to interpret and incur more training and test time as compared to smaller trees.

In the following, we present different ways of measuring the impurity of a node and the collective impurity of its child nodes, both of which will be used to identify the best attribute test condition for a node.

Impurity Measure for a Single Node

The impurity of a node measures how dissimilar the class labels are for the data instances belonging to a common node. Following are examples of measures

that can be used to evaluate the impurity of a node t:

$$\text{Entropy} = -\sum_{i=0}^{c-1} p_i(t) \log_2 p_i(t), \tag{3.4}$$

$$\text{Gini index} = 1 - \sum_{i=0}^{c-1} p_i(t)^2, \tag{3.5}$$

$$\text{Classification error} = 1 - \max_i [p_i(t)], \tag{3.6}$$

where $p_i(t)$ is the relative frequency of training instances that belong to class i at node t, c is the total number of classes, and $0 \log_2 0 = 0$ in entropy calculations. All three measures give a zero impurity value if a node contains instances from a single class and maximum impurity if the node has equal proportion of instances from multiple classes.

Figure 3.11 compares the relative magnitude of the impurity measures when applied to binary classification problems. Since there are only two classes, $p_0(t) + p_1(t) = 1$. The horizontal axis p refers to the fraction of instances that belong to one of the two classes. Observe that all three measures attain their maximum value when the class distribution is uniform (i.e., $p_0(t) = p_1(t) = 0.5$) and minimum value when all the instances belong to a single class (i.e., either $p_0(t)$ or $p_1(t)$ equals to 1). The following examples illustrate how the values of the impurity measures vary as we alter the class distribution.

Node N_1	Count
Class=0	0
Class=1	6

Gini $= 1 - (0/6)^2 - (6/6)^2 = 0$
Entropy $= -(0/6)\log_2(0/6) - (6/6)\log_2(6/6) = 0$
Error $= 1 - \max[0/6, 6/6] = 0$

Node N_2	Count
Class=0	1
Class=1	5

Gini $= 1 - (1/6)^2 - (5/6)^2 = 0.278$
Entropy $= -(1/6)\log_2(1/6) - (5/6)\log_2(5/6) = 0.650$
Error $= 1 - \max[1/6, 5/6] = 0.167$

Node N_3	Count
Class=0	3
Class=1	3

Gini $= 1 - (3/6)^2 - (3/6)^2 = 0.5$
Entropy $= -(3/6)\log_2(3/6) - (3/6)\log_2(3/6) = 1$
Error $= 1 - \max[3/6, 3/6] = 0.5$

Based on these calculations, node N_1 has the lowest impurity value, followed by N_2 and N_3. This example, along with Figure 3.11, shows the consistency among the impurity measures, i.e., if a node N_1 has lower entropy than node N_2, then the Gini index and error rate of N_1 will also be lower than that

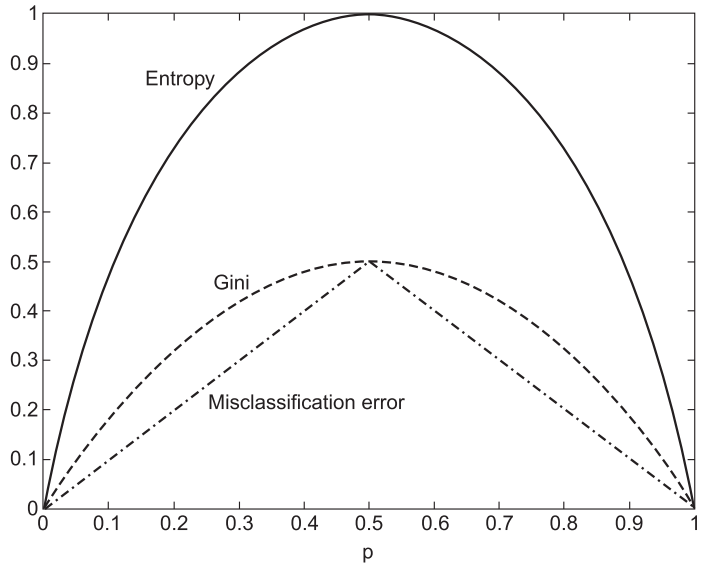


Figure 3.11. Comparison among the impurity measures for binary classification problems.

of N_2. Despite their agreement, the attribute chosen as splitting criterion by the impurity measures can still be different (see Exercise 7 on pages 206–207).

Collective Impurity of Child Nodes

Consider an attribute test condition that splits a node containing N training instances into k children, $\{v_1, v_2, \cdots, v_k\}$, where every child node represents a partition of the data resulting from one of the k outcomes of the attribute test condition. Let $N(v_j)$ be the number of training instances associated with a child node v_j, whose impurity value is $I(v_j)$. Since a training instance in the parent node reaches node v_j for a fraction of $N(v_j)/N$ times, the collective impurity of the child nodes can be computed by taking a weighted sum of the impurities of the child nodes, as follows:

$$I(\text{children}) = \sum_{j=1}^{k} \frac{N(v_j)}{N} I(v_j), \tag{3.7}$$

Example 3.3. [Weighted Entropy] Consider the candidate attribute test condition shown in Figures 3.12(a) and (b) for the loan borrower classification problem. Splitting on the `Home Owner` attribute will generate two child nodes

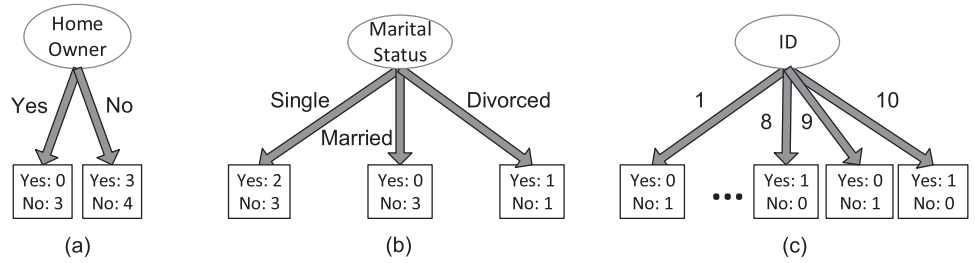


Figure 3.12. Examples of candidate attribute test conditions.

whose weighted entropy can be calculated as follows:

$$I(\texttt{Home Owner = yes}) = -\frac{0}{3}\log_2\frac{0}{3} - \frac{3}{3}\log_2\frac{3}{3} = 0$$
$$I(\texttt{Home Owner = no}) = -\frac{3}{7}\log_2\frac{3}{7} - \frac{4}{7}\log_2\frac{4}{7} = 0.985$$
$$I(\texttt{Home Owner}) = \frac{3}{10}\times 0 + \frac{7}{10}\times 0.985 = 0.690$$

Splitting on `Marital Status`, on the other hand, leads to three child nodes with a weighted entropy given by

$$I(\texttt{Marital Status = Single}) = -\frac{2}{5}\log_2\frac{2}{5} - \frac{3}{5}\log_2\frac{3}{5} = 0.971$$
$$I(\texttt{Marital Status = Married}) = -\frac{0}{3}\log_2\frac{0}{3} - \frac{3}{3}\log_2\frac{3}{3} = 0$$
$$I(\texttt{Marital Status = Divorced}) = -\frac{1}{2}\log_2\frac{1}{2} - \frac{1}{2}\log_2\frac{1}{2} = 1.000$$
$$I(\texttt{Marital Status}) = \frac{5}{10}\times 0.971 + \frac{3}{10}\times 0 + \frac{2}{10}\times 1 = 0.686$$

Thus, `Marital Status` has a lower weighted entropy than `Home Owner`. ■

Identifying the best attribute test condition

To determine the goodness of an attribute test condition, we need to compare the degree of impurity of the parent node (before splitting) with the weighted degree of impurity of the child nodes (after splitting). The larger their difference, the better the test condition. This difference, Δ, also termed as the **gain** in purity of an attribute test condition, can be defined as follows:

$$\Delta = I(\text{parent}) - I(\text{children}), \tag{3.8}$$

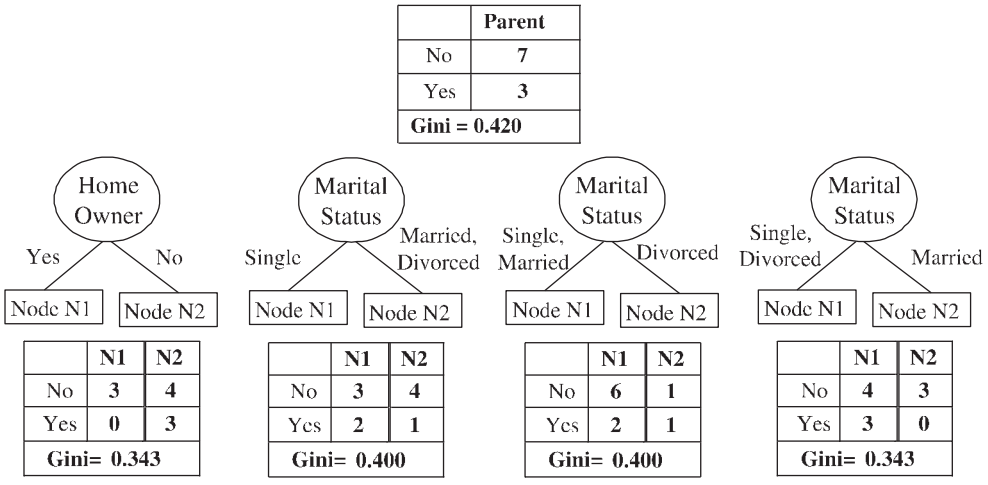


Figure 3.13. Splitting criteria for the loan borrower classification problem using Gini index.

where $I(\text{parent})$ is the impurity of a node before splitting and $I(\text{children})$ is the weighted impurity measure after splitting. It can be shown that the gain is non-negative since $I(\text{parent}) \geq I(\text{children})$ for any reasonable measure such as those presented above. The higher the gain, the purer are the classes in the child nodes relative to the parent node. The splitting criterion in the decision tree learning algorithm selects the attribute test condition that shows the maximum gain. Note that maximizing the gain at a given node is equivalent to minimizing the weighted impurity measure of its children since $I(\text{parent})$ is the same for all candidate attribute test conditions. Finally, when entropy is used as the impurity measure, the difference in entropy is commonly known as **information gain**, Δ_{info}.

In the following, we present illustrative approaches for identifying the best attribute test condition given qualitative or quantitative attributes.

Splitting of Qualitative Attributes

Consider the first two candidate splits shown in Figure 3.12 involving qualitative attributes `Home Owner` and `Marital Status`. The initial class distribution at the parent node is $(0.3, 0.7)$, since there are 3 instances of class `Yes` and 7 instances of class `No` in the training data. Thus,

$$I(\text{parent}) = -\frac{3}{10}\log_2\frac{3}{10} - \frac{7}{10}\log_2\frac{7}{10} = 0.881$$

The information gains for `Home Owner` and `Marital Status` are each given by

$$\begin{aligned} \Delta_{\text{info}}(\texttt{Home Owner}) &= 0.881 - 0.690 = 0.191 \\ \Delta_{\text{info}}(\texttt{Marital Status}) &= 0.881 - 0.686 = 0.195 \end{aligned}$$

The information gain for `Marital Status` is thus higher due to its lower weighted entropy, which will thus be considered for splitting.

Binary Splitting of Qualitative Attributes

Consider building a decision tree using only binary splits and the Gini index as the impurity measure. Figure 3.13 shows examples of four candidate splitting criteria for the `Home Owner` and `Marital Status` attributes. Since there are 3 borrowers in the training set who defaulted and 7 others who repaid their loan (see Table in Figure 3.13), the Gini index of the parent node before splitting is

$$1 - \left(\frac{3}{10}\right)^2 - \left(\frac{7}{10}\right)^2 = 0.420.$$

If `Home Owner` is chosen as the splitting attribute, the Gini index for the child nodes $\texttt{N}_1$ and $\texttt{N}_2$ are 0 and 0.490, respectively. The weighted average Gini index for the children is

$$(3/10) \times 0 + (7/10) \times 0.490 = 0.343,$$

where the weights represent the proportion of training instances assigned to each child. The gain using `Home Owner` as splitting attribute is 0.420 - 0.343 = 0.077. Similarly, we can apply a binary split on the `Marital Status` attribute. However, since `Marital Status` is a nominal attribute with three outcomes, there are three possible ways to group the attribute values into a binary split. The weighted average Gini index of the children for each candidate binary split is shown in Figure 3.13. Based on these results, `Home Owner` and the last binary split using `Marital Status` are clearly the best candidates, since they both produce the lowest weighted average Gini index. Binary splits can also be used for ordinal attributes, if the binary partitioning of the attribute values does not violate the ordering property of the values.

Binary Splitting of Quantitative Attributes

Consider the problem of identifying the best binary split $\texttt{Annual Income} \leq \tau$ for the preceding loan approval classification problem. As discussed previously,

Class	No	No	No	Yes	Yes	Yes	No	No	No	No
	Annual Income (in '000s)									
Sorted Values →	60	70	75	85	90	95	100	120	125	220
Split Positions →	55	65	72.5	80	87.5	92.5	97.5	110	122.5	172.5	230
	<= >	<= >	<= >	<= >	<= >	<= >	<= >	<= >	<= >	<= >	<= >
Yes	0 3	0 3	0 3	0 3	1 2	2 1	3 0	3 0	3 0	3 0	3 0
No	0 7	1 6	2 5	3 4	3 4	3 4	3 4	4 3	5 2	6 1	7 0
Gini	0.420	0.400	0.375	0.343	0.417	0.400	***0.300***	0.343	0.375	0.400	0.420

Figure 3.14. Splitting continuous attributes.

even though τ can take any value between the minimum and maximum values of annual income in the training set, it is sufficient to only consider the annual income values observed in the training set as candidate split positions. For each candidate τ, the training set is scanned once to count the number of borrowers with annual income less than or greater than τ along with their class proportions. We can then compute the Gini index at each candidate split position and choose the τ that produces the lowest value. Computing the Gini index at each candidate split position requires $O(N)$ operations, where N is the number of training instances. Since there are at most N possible candidates, the overall complexity of this brute-force method is $O(N^2)$. It is possible to reduce the complexity of this problem to $O(N \log N)$ by using a method described as follows (see illustration in Figure 3.14). In this method, we first sort the training instances based on their annual income, a one-time cost that requires $O(N \log N)$ operations. The candidate split positions are given by the midpoints between every two adjacent sorted values: \$55,000, \$65,000, \$72,500, and so on. For the first candidate, since none of the instances has an annual income less than or equal to \$55,000, the Gini index for the child node with `Annual Income` $<$ \$55,000 is equal to zero. In contrast, there are 3 training instances of class `Yes` and 7 instances of class `No` with annual income greater than \$55,000. The Gini index for this node is 0.420. The weighted average Gini index for the first candidate split position, $\tau = \$55,000$, is equal to $0 \times 0 + 1 \times 0.420 = 0.420$.

For the next candidate, $\tau = \$65,000$, the class distribution of its child nodes can be obtained with a simple update of the distribution for the previous candidate. This is because, as τ increases from \$55,000 to \$65,000, there is only one training instance affected by the change. By examining the class label of the affected training instance, the new class distribution is obtained. For example, as τ increases to \$65,000, there is only one borrower in the training

set, with an annual income of \$60,000, affected by this change. Since the class label for the borrower is `No`, the count for class `No` increases from 0 to 1 (for `Annual Income` $\leq$ \$65,000) and decreases from 7 to 6 (for `Annual Income` $>$ \$65,000), as shown in Figure 3.14. The distribution for the `Yes` class remains unaffected. The updated Gini index for this candidate split position is 0.400.

This procedure is repeated until the Gini index for all candidates are found. The best split position corresponds to the one that produces the lowest Gini index, which occurs at $\tau = \$97,500$. Since the Gini index at each candidate split position can be computed in $O(1)$ time, the complexity of finding the best split position is $O(N)$ once all the values are kept sorted, a one-time operation that takes $O(N \log N)$ time. The overall complexity of this method is thus $O(N \log N)$, which is much smaller than the $O(N^2)$ time taken by the brute-force method. The amount of computation can be further reduced by considering only candidate split positions located between two adjacent sorted instances with different class labels. For example, we do not need to consider candidate split positions located between \$60,000 and \$75,000 because all three instances with annual income in this range (\$60,000, \$70,000, and \$75,000) have the same class labels. Choosing a split position within this range only increases the degree of impurity, compared to a split position located outside this range. Therefore, the candidate split positions at $\tau = \$65,000$ and $\tau = \$72,500$ can be ignored. Similarly, we do not need to consider the candidate split positions at \$87,500, \$92,500, \$110,000, \$122,500, and \$172,500 because they are located between two adjacent instances with the same labels. This strategy reduces the number of candidate split positions to consider from 9 to 2 (excluding the two boundary cases $\tau = \$55,000$ and $\tau = \$230,000$).

Gain Ratio

One potential limitation of impurity measures such as entropy and Gini index is that they tend to favor qualitative attributes with large number of distinct values. Figure 3.12 shows three candidate attributes for partitioning the data set given in Table 3.3. As previously mentioned, the attribute `Marital Status` is a better choice than the attribute `Home Owner`, because it provides a larger information gain. However, if we compare them against `Customer ID`, the latter produces the purest partitions with the maximum information gain, since the weighted entropy and Gini index is equal to zero for its children. Yet, `Customer ID` is not a good attribute for splitting because it has a unique value for each instance. Even though a test condition involving `Customer ID` will accurately classify every instance in the training data, we cannot use such a test condition on new test instances with `Customer ID` values that haven't

been seen before during training. This example suggests having a low impurity value alone is insufficient to find a good attribute test condition for a node. As we will see later in Section 3.4, having more number of child nodes can make a decision tree more complex and consequently more susceptible to overfitting. Hence, the number of children produced by the splitting attribute should also be taken into consideration while deciding the best attribute test condition.

There are two ways to overcome this problem. One way is to generate only binary decision trees, thus avoiding the difficulty of handling attributes with varying number of partitions. This strategy is employed by decision tree classifiers such as CART. Another way is to modify the splitting criterion to take into account the number of partitions produced by the attribute. For example, in the C4.5 decision tree algorithm, a measure known as **gain ratio** is used to compensate for attributes that produce a large number of child nodes. This measure is computed as follows:

$$\text{Gain ratio} = \frac{\Delta_{\text{info}}}{\text{Split Info}} = \frac{\text{Entropy(Parent)} - \sum_{i=1}^{k} \frac{N(v_i)}{N} \text{Entropy}(v_i)}{-\sum_{i=1}^{k} \frac{N(v_i)}{N} \log_2 \frac{N(v_i)}{N}} \quad (3.9)$$

where $N(v_i)$ is the number of instances assigned to node v_i and k is the total number of splits. The split information measures the entropy of splitting a node into its child nodes and evaluates if the split results in a larger number of equally-sized child nodes or not. For example, if every partition has the same number of instances, then $\forall i : N(v_i)/N = 1/k$ and the split information would be equal to $\log_2 k$. Thus, if an attribute produces a large number of splits, its split information is also large, which in turn, reduces the gain ratio.

Example 3.4. [Gain Ratio] Consider the data set given in Exercise 3 on page 205. We want to select the best attribute test condition among the following three attributes: `Gender`, `Car Type`, and `Customer ID`. The entropy before splitting is

$$\text{Entropy(parent)} = -\frac{10}{20}\log_2\frac{10}{20} - \frac{10}{20}\log_2\frac{10}{20} = 1.$$

If `Gender` is used as attribute test condition:

$$\begin{aligned} \text{Entropy(children)} &= \frac{10}{20}\left[-\frac{6}{10}\log_2\frac{6}{10} - \frac{4}{10}\log_2\frac{4}{10}\right] \times 2 = 0.971 \\ \text{Gain Ratio} &= \frac{1 - 0.971}{-\frac{10}{20}\log_2\frac{10}{20} - \frac{10}{20}\log_2\frac{10}{20}} = \frac{0.029}{1} = 0.029 \end{aligned}$$

If `Car Type` is used as attribute test condition:

$$\begin{aligned}\text{Entropy(children)} &= \frac{4}{20}\left[-\frac{1}{4}\log_2\frac{1}{4}-\frac{3}{4}\log_2\frac{3}{4}\right]+\frac{8}{20}\times 0 \\ &\quad + \frac{8}{20}\left[-\frac{1}{8}\log_2\frac{1}{8}-\frac{7}{8}\log_2\frac{7}{8}\right] = 0.380 \\ \text{Gain Ratio} &= \frac{1-0.380}{-\frac{4}{20}\log_2\frac{4}{20}-\frac{8}{20}\log_2\frac{8}{20}-\frac{8}{20}\log_2\frac{8}{20}} = \frac{0.620}{1.52} = 0.41\end{aligned}$$

Finally, if `Customer ID` is used as attribute test condition:

$$\begin{aligned}\text{Entropy(children)} &= \frac{1}{20}\left[-\frac{1}{1}\log_2\frac{1}{1}-\frac{0}{1}\log_2\frac{0}{1}\right]\times 20 = 0 \\ \text{Gain Ratio} &= \frac{1-0}{-\frac{1}{20}\log_2\frac{1}{20}\times 20} = \frac{1}{4.32} = 0.23\end{aligned}$$

Thus, even though `Customer ID` has the highest information gain, its gain ratio is lower than `Car Type` since it produces a larger number of splits. ■

3.3.4 Algorithm for Decision Tree Induction

Algorithm 3.1 presents a pseudocode for decision tree induction algorithm. The input to this algorithm is a set of training instances E along with the attribute set F. The algorithm works by recursively selecting the best attribute to split the data (Step 7) and expanding the nodes of the tree (Steps 11 and 12) until the stopping criterion is met (Step 1). The details of this algorithm are explained below.

1. The `createNode`() function extends the decision tree by creating a new node. A node in the decision tree either has a test condition, denoted as *node.test_cond*, or a class label, denoted as *node.label*.

2. The `find_best_split`() function determines the attribute test condition for partitioning the training instances associated with a node. The splitting attribute chosen depends on the impurity measure used. The popular measures include entropy and the Gini index.

3. The `Classify`() function determines the class label to be assigned to a leaf node. For each leaf node t, let $p(i|t)$ denote the fraction of training instances from class i associated with the node t. The label assigned

Algorithm 3.1 A skeleton decision tree induction algorithm.

```
TreeGrowth (E, F)
 1: if stopping_cond(E,F) = true then
 2:    leaf = createNode().
 3:    leaf.label = Classify(E).
 4:    return leaf.
 5: else
 6:    root = createNode().
 7:    root.test_cond = find_best_split(E, F).
 8:    let V = {v|v is a possible outcome of root.test_cond }.
 9:    for each v ∈ V do
10:       E_v = {e | root.test_cond(e) = v and e ∈ E}.
11:       child = TreeGrowth(E_v, F).
12:       add child as descendent of root and label the edge (root → child) as v.
13:    end for
14: end if
15: return root.
```

to the leaf node is typically the one that occurs most frequently in the training instances that are associated with this node.

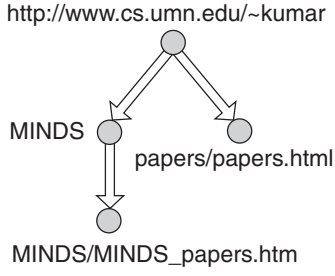
$$leaf.label = \underset{i}{\text{argmax}}\ p(i|t), \tag{3.10}$$

where the argmax operator returns the class i that maximizes $p(i|t)$. Besides providing the information needed to determine the class label of a leaf node, $p(i|t)$ can also be used as a rough estimate of the probability that an instance assigned to the leaf node t belongs to class i. Sections 6.11.2 and 6.11.4 in the next chapter describe how such probability estimates can be used to determine the performance of a decision tree under different cost functions.

4. The `stopping_cond`() function is used to terminate the tree-growing process by checking whether all the instances have identical class label or attribute values. Since decision tree classifiers employ a top-down, recursive partitioning approach for building a model, the number of training instances associated with a node decreases as the depth of the tree increases. As a result, a leaf node may contain too few training instances to make a statistically significant decision about its class label. This is known as the **data fragmentation** problem. One way to avoid this problem is to disallow splitting of a node when the number of instances associated with the node fall below a certain threshold. A

Session	IP Address	Timestamp	Request Method	Requested Web Page	Protocol	Status	Number of Bytes	Referrer	User Agent
1	160.11.11.11	08/Aug/2004 10:15:21	GET	http://www.cs.umn.edu/~kumar	HTTP/1.1	200	6424		Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.0)
1	160.11.11.11	08/Aug/2004 10:15:34	GET	http://www.cs.umn.edu/~kumar/MINDS	HTTP/1.1	200	41378	http://www.cs.umn.edu/~kumar	Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.0)
1	160.11.11.11	08/Aug/2004 10:15:41	GET	http://www.cs.umn.edu/~kumar/MINDS/MINDS_papers.htm	HTTP/1.1	200	1018516	http://www.cs.umn.edu/~kumar/MINDS	Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.0)
1	160.11.11.11	08/Aug/2004 10:16:11	GET	http://www.cs.umn.edu/~kumar/papers/papers.html	HTTP/1.1	200	7463	http://www.cs.umn.edu/~kumar	Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.0)
2	35.9.2.2	08/Aug/2004 10:16:15	GET	http://www.cs.umn.edu/~steinbac	HTTP/1.0	200	3149		Mozilla/5.0 (Windows; U; Windows NT 5.1; en-US; rv:1.7) Gecko/20040616

(a) Example of a Web server log.



(b) Graph of a Web session.

Attribute Name	Description
totalPages	Total number of pages retrieved in a Web session
ImagePages	Total number of image pages retrieved in a Web session
TotalTime	Total amount of time spent by Web site visitor
RepeatedAccess	The same page requested more than once in a Web session
ErrorRequest	Errors in requesting for Web pages
GET	Percentage of requests made using GET method
POST	Percentage of requests made using POST method
HEAD	Percentage of requests made using HEAD method
Breadth	Breadth of Web traversal
Depth	Depth of Web traversal
MultiIP	Session with multiple IP addresses
MultiAgent	Session with multiple user agents

(c) Derived attributes for Web robot detection.

Figure 3.15. Input data for web robot detection.

more systematic way to control the size of a decision tree (number of leaf nodes) will be discussed in Section 3.5.4.

3.3.5 Example Application: Web Robot Detection

Consider the task of distinguishing the access patterns of web robots from those generated by human users. A web robot (also known as a web crawler) is a software program that automatically retrieves files from one or more websites by following the hyperlinks extracted from an initial set of seed URLs. These programs have been deployed for various purposes, from gathering web pages on behalf of search engines to more malicious activities such as spamming and committing click frauds in online advertisements.

The web robot detection problem can be cast as a binary classification task. The input data for the classification task is a web server log, a sample of which is shown in Figure 3.15(a). Each line in the log file corresponds to a

request made by a client (i.e., a human user or a web robot) to the web server. The fields recorded in the web log include the client's IP address, timestamp of the request, URL of the requested file, size of the file, and **user agent**, which is a field that contains identifying information about the client. For human users, the user agent field specifies the type of web browser or mobile device used to fetch the files, whereas for web robots, it should technically contain the name of the crawler program. However, web robots may conceal their true identities by declaring their user agent fields to be identical to known browsers. Therefore, user agent is not a reliable field to detect web robots.

The first step toward building a classification model is to precisely define a data instance and associated attributes. A simple approach is to consider each log entry as a data instance and use the appropriate fields in the log file as its attribute set. This approach, however, is inadequate for several reasons. First, many of the attributes are nominal-valued and have a wide range of domain values. For example, the number of unique client IP addresses, URLs, and referrers in a log file can be very large. These attributes are undesirable for building a decision tree because their split information is extremely high (see Equation (3.9)). In addition, it might not be possible to classify test instances containing IP addresses, URLs, or referrers that are not present in the training data. Finally, by considering each log entry as a separate data instance, we disregard the sequence of web pages retrieved by the client—a critical piece of information that can help distinguish web robot accesses from those of a human user.

A better alternative is to consider each web session as a data instance. A web session is a sequence of requests made by a client during a given visit to the website. Each web session can be modeled as a directed graph, in which the nodes correspond to web pages and the edges correspond to hyperlinks connecting one web page to another. Figure 3.15(b) shows a graphical representation of the first web session given in the log file. Every web session can be characterized using some meaningful attributes about the graph that contain discriminatory information. Figure 3.15(c) shows some of the attributes extracted from the graph, including the `depth` and `breadth` of its corresponding tree rooted at the entry point to the website. For example, the `depth` and `breadth` of the tree shown in Figure 3.15(b) are both equal to two.

The derived attributes shown in Figure 3.15(c) are more informative than the original attributes given in the log file because they characterize the behavior of the client at the website. Using this approach, a data set containing 2916 instances was created, with equal numbers of sessions due to web robots (class 1) and human users (class 0). 10% of the data were reserved for training while the remaining 90% were used for testing. The induced decision tree is

shown in Figure 3.16, which has an error rate equal to 3.8% on the training set and 5.3% on the test set. In addition to its low error rate, the tree also reveals some interesting properties that can help discriminate web robots from human users:

1. Accesses by web robots tend to be broad but shallow, whereas accesses by human users tend to be more focused (narrow but deep).

2. Web robots seldom retrieve the image pages associated with a web page.

3. Sessions due to web robots tend to be long and contain a large number of requested pages.

4. Web robots are more likely to make repeated requests for the same web page than human users since the web pages retrieved by human users are often cached by the browser.

3.3.6 Characteristics of Decision Tree Classifiers

The following is a summary of the important characteristics of decision tree induction algorithms.

1. **Applicability:** Decision trees are a nonparametric approach for building classification models. This approach does not require any prior assumption about the probability distribution governing the class and attributes of the data, and thus, is applicable to a wide variety of data sets. It is also applicable to both categorical and continuous data without requiring the attributes to be transformed into a common representation via binarization, normalization, or standardization. Unlike some binary classifiers described in Chapter 6, it can also deal with multiclass problems without the need to decompose them into multiple binary classification tasks. Another appealing feature of decision tree classifiers is that the induced trees, especially the shorter ones, are relatively easy to interpret. The accuracies of the trees are also quite comparable to other classification techniques for many simple data sets.

2. **Expressiveness:** A decision tree provides a universal representation for discrete-valued functions. In other words, it can encode any function of discrete-valued attributes. This is because every discrete-valued function can be represented as an assignment table, where every unique combination of discrete attributes is assigned a class label. Since every

```
Decision Tree:
depth = 1:
| breadth> 7 : class 1
| breadth<= 7:
| | breadth <= 3:
| | | ImagePages> 0.375: class 0
| | | ImagePages<= 0.375:
| | | | totalPages<= 6: class 1
| | | | totalPages> 6:
| | | | | breadth <= 1: class 1
| | | | | breadth > 1: class 0
| | width > 3:
| | | MultiIP = 0:
| | | | ImagePages<= 0.1333: class 1
| | | | ImagePages> 0.1333:
| | | | breadth <= 6: class 0
| | | | breadth > 6: class 1
| | | MultiIP = 1:
| | | | TotalTime <= 361: class 0
| | | | TotalTime > 361: class 1
depth> 1:
| MultiAgent = 0:
| | depth > 2: class 0
| | depth < 2:
| | | MultiIP = 1: class 0
| | | MultiIP = 0:
| | | | breadth <= 6: class 0
| | | | breadth > 6:
| | | | | RepeatedAccess <= 0.322: class 0
| | | | | RepeatedAccess > 0.322: class 1
| MultiAgent = 1:
| | totalPages <= 81: class 0
| | totalPages > 81: class 1
```

Figure 3.16. Decision tree model for web robot detection.

combination of attributes can be represented as a leaf in the decision tree, we can always find a decision tree whose label assignments at the leaf nodes matches with the assignment table of the original function. Decision trees can also help in providing compact representations of functions when some of the unique combinations of attributes can be represented by the same leaf node. For example, Figure 3.17 shows the assignment table of the Boolean function $(A \wedge B) \vee (C \wedge D)$ involving four binary attributes, resulting in a total of $2^4 = 16$ possible assignments. The tree shown in Figure 3.17 shows a compressed encoding of this assignment table. Instead of requiring a fully-grown tree with 16 leaf nodes, it is possible to encode the function using a simpler tree with only 7 leaf nodes. Nevertheless, not all decision trees for discrete-valued attributes can be simplified. One notable example is the parity function,

A	B	C	D	class
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

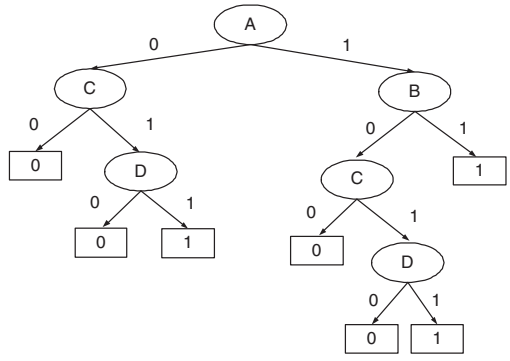


Figure 3.17. Decision tree for the Boolean function $(A \wedge B) \vee (C \wedge D)$.

whose value is 1 when there is an even number of true values among its Boolean attributes, and 0 otherwise. Accurate modeling of such a function requires a full decision tree with 2^d nodes, where d is the number of Boolean attributes (see Exercise 2 on page 205).

3. **Computational Efficiency:** Since the number of possible decision trees can be very large, many decision tree algorithms employ a heuristic-based approach to guide their search in the vast hypothesis space. For example, the algorithm presented in Section 3.3.4 uses a greedy, top-down, recursive partitioning strategy for growing a decision tree. For many data sets, such techniques quickly construct a reasonably good decision tree even when the training set size is very large. Furthermore, once a decision tree has been built, classifying a test record is extremely fast, with a worst-case complexity of $O(w)$, where w is the maximum depth of the tree.

4. **Handling Missing Values:** A decision tree classifier can handle missing attribute values in a number of ways, both in the training and the test sets. When there are missing values in the test set, the classifier must decide which branch to follow if the value of a splitting node attribute is missing for a given test instance. One approach, known as the **probabilistic split method**, which is employed by the C4.5 decision tree classifier, distributes the data instance to every child of the splitting node according to the probability that the missing attribute has a particular value. In contrast, the CART algorithm uses the **surrogate split method**, where the instance whose splitting attribute value is

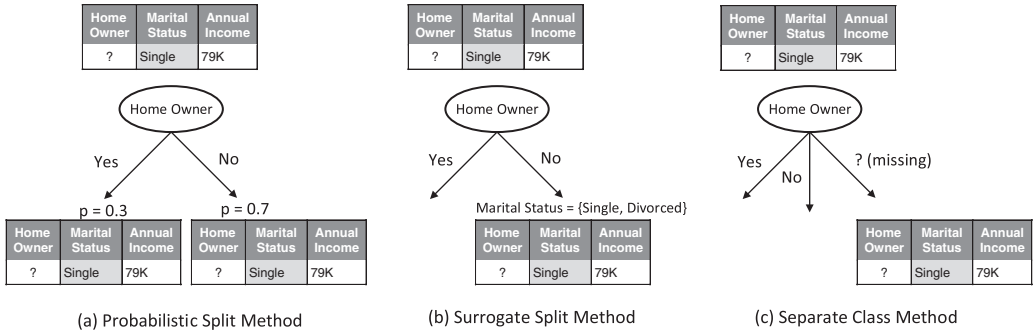


Figure 3.18. Methods for handling missing attribute values in decision tree classifier.

missing is assigned to one of the child nodes based on the value of another non-missing surrogate attribute whose splits most resemble the partitions made by the missing attribute. Another approach, known as the **separate class method** is used by the CHAID algorithm, where the missing value is treated as a separate categorical value distinct from other values of the splitting attribute. Figure 3.18 shows an example of the three different ways for handling missing values in a decision tree classifier. Other strategies for dealing with missing values are based on data preprocessing, where the instance with missing value is either imputed with the mode (for categorical attribute) or mean (for continuous attribute) value or discarded before the classifier is trained.

During training, if an attribute v has missing values in some of the training instances associated with a node, we need a way to measure the gain in purity if v is used for splitting. One simple way is to exclude instances with missing values of v in the counting of instances associated with every child node, generated for every possible outcome of v. Further, if v is chosen as the attribute test condition at a node, training instances with missing values of v can be propagated to the child nodes using any of the methods described above for handling missing values in test instances.

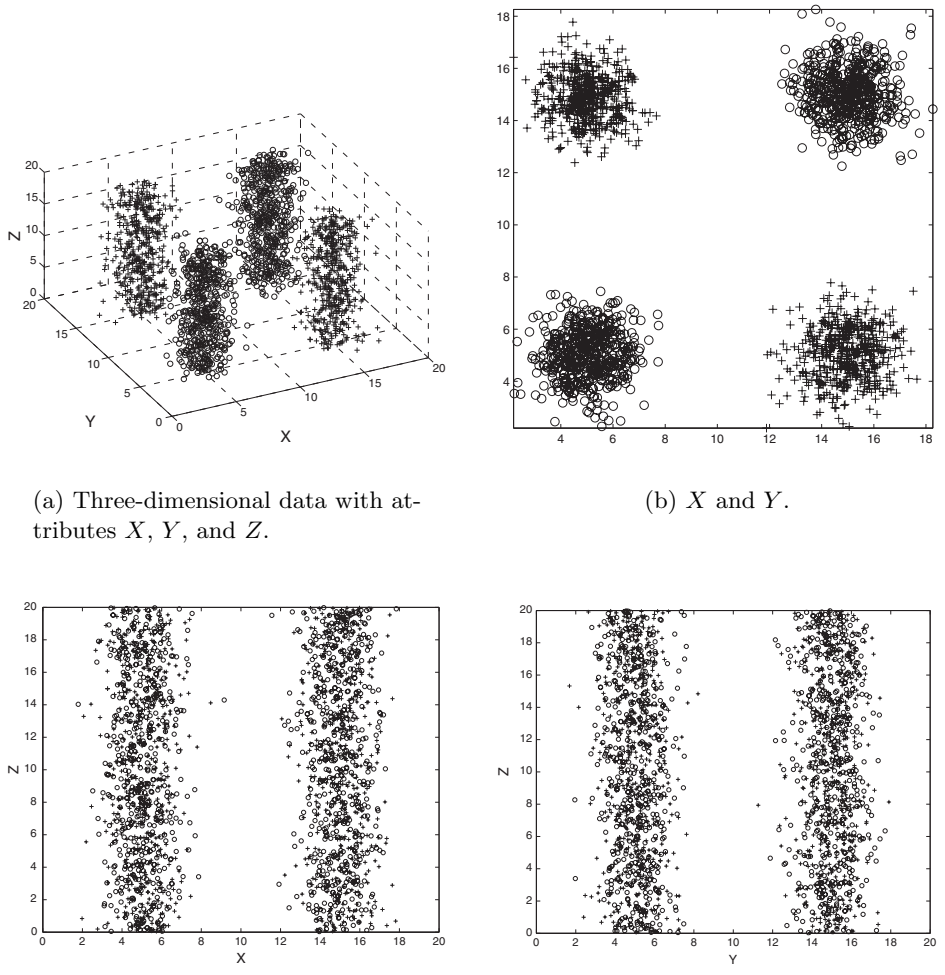
5. **Handling Interactions among Attributes:** Attributes are considered interacting if they are able to distinguish between classes when used together, but individually they provide little or no information. Due to the greedy nature of the splitting criteria in decision trees, such attributes could be passed over in favor of other attributes that are not as useful. This could result in more complex decision trees than necessary.

Hence, decision trees can perform poorly when there are interactions among attributes.

To illustrate this point, consider the three-dimensional data shown in Figure 3.19(a), which contains 2000 data points from one of two classes, denoted as + and ∘ in the diagram. Figure 3.19(b) shows the distribution of the two classes in the two-dimensional space involving attributes X and Y, which is a noisy version of the XOR Boolean function. We can see that even though the two classes are well-separated in this two-dimensional space, neither of the two attributes contain sufficient information to distinguish between the two classes when used alone. For example, the entropies of the following attribute test conditions: $X \leq 10$ and $Y \leq 10$, are close to 1, indicating that neither X nor Y provide any reduction in the impurity measure when used individually. X and Y thus represent a case of interaction among attributes. The data set also contains a third attribute, Z, in which both classes are distributed uniformly, as shown in Figures 3.19(c) and 3.19(d), and hence, the entropy of any split involving Z is close to 1. As a result, Z is as likely to be chosen for splitting as the interacting but useful attributes, X and Y. For further illustration of this issue, readers are referred to Example 3.7 in Section 3.4.1 and Exercise 8 at the end of this chapter.

6. **Handling Irrelevant Attributes:** An attribute is irrelevant if it is not useful for the classification task. Since irrelevant attributes are poorly associated with the target class labels, they will provide little or no gain in purity and thus will be passed over by other more relevant features. Hence, the presence of a small number of irrelevant attributes will not impact the decision tree construction process. However, not all attributes that provide little to no gain are irrelevant (see Figure 3.19). Hence, if the classification problem is complex (e.g., involving interactions among attributes) and there are a large number of irrelevant attributes, then some of these attributes may be accidentally chosen during the tree-growing process, since they may provide a better gain than a relevant attribute just by random chance. Feature selection techniques can help to improve the accuracy of decision trees by eliminating the irrelevant attributes during preprocessing. We will investigate the issue of too many irrelevant attributes in Section 3.4.1.

7. **Handling Redundant Attributes:** An attribute is redundant if it is strongly correlated with another attribute in the data. Since redundant



(a) Three-dimensional data with attributes X, Y, and Z.

(b) X and Y.

(c) X and Z.

(d) Y and Z.

Figure 3.19. Example of a XOR data involving X and Y, along with an irrelevant attribute Z.

attributes show similar gains in purity if they are selected for splitting, only one of them will be selected as an attribute test condition in the decision tree algorithm. Decision trees can thus handle the presence of redundant attributes.

8. **Using Rectilinear Splits:** The test conditions described so far in this chapter involve using only a single attribute at a time. As a consequence,

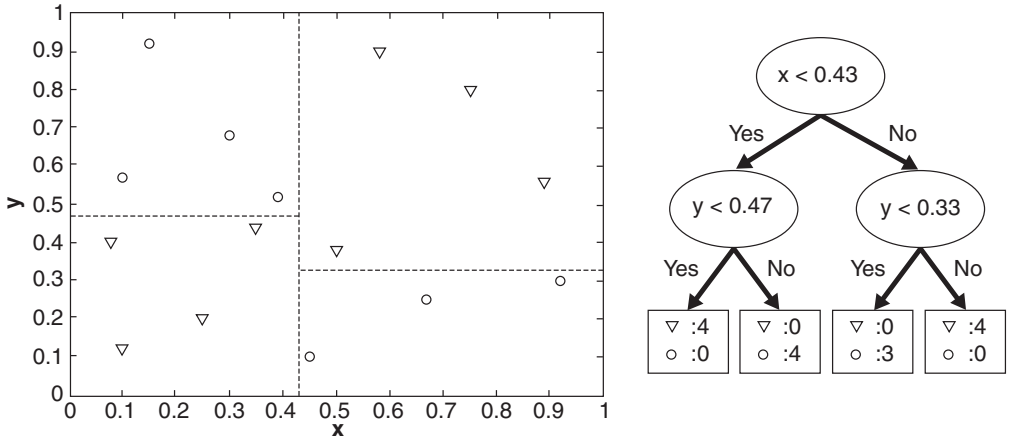


Figure 3.20. Example of a decision tree and its decision boundaries for a two-dimensional data set.

the tree-growing procedure can be viewed as the process of partitioning the attribute space into disjoint regions until each region contains records of the same class. The border between two neighboring regions of different classes is known as a **decision boundary**. Figure 3.20 shows the decision tree as well as the decision boundary for a binary classification problem. Since the test condition involves only a single attribute, the decision boundaries are rectilinear; i.e., parallel to the coordinate axes. This limits the expressiveness of decision trees in representing decision boundaries of data sets with continuous attributes. Figure 3.21 shows a two-dimensional data set involving binary classes that cannot be perfectly classified by a decision tree whose attribute test conditions are defined based on single attributes. The binary classes in the data set are generated from two skewed Gaussian distributions, centered at (8,8) and (12,12), respectively. The true decision boundary is represented by the diagonal dashed line, whereas the rectilinear decision boundary produced by the decision tree classifier is shown by the thick solid line.

In contrast, an **oblique decision tree** may overcome this limitation by allowing the test condition to be specified using more than one attribute. For example, the binary classification data shown in Figure 3.21 can be easily represented by an oblique decision tree with a single root node with test condition

$$x + y < 20.$$

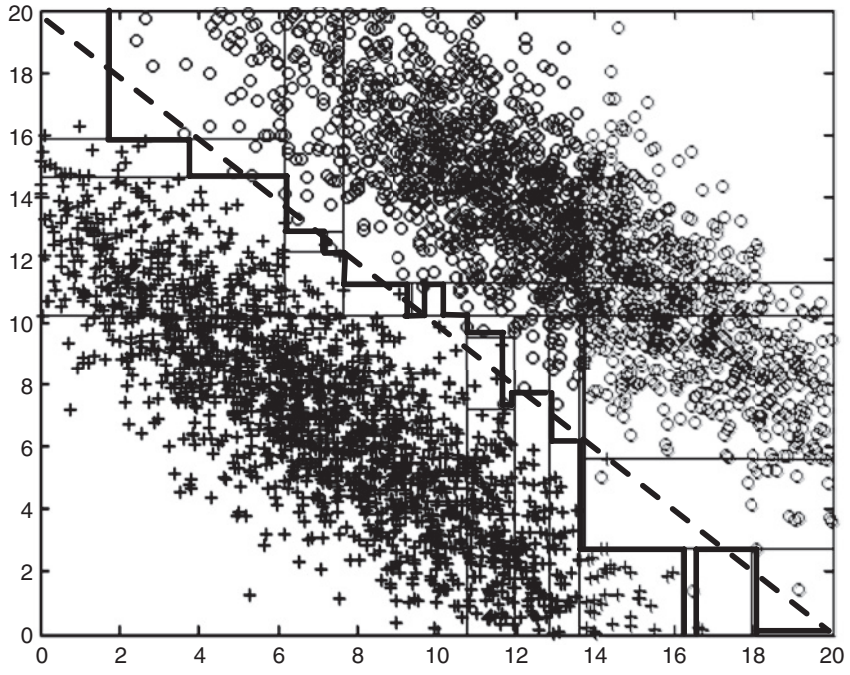


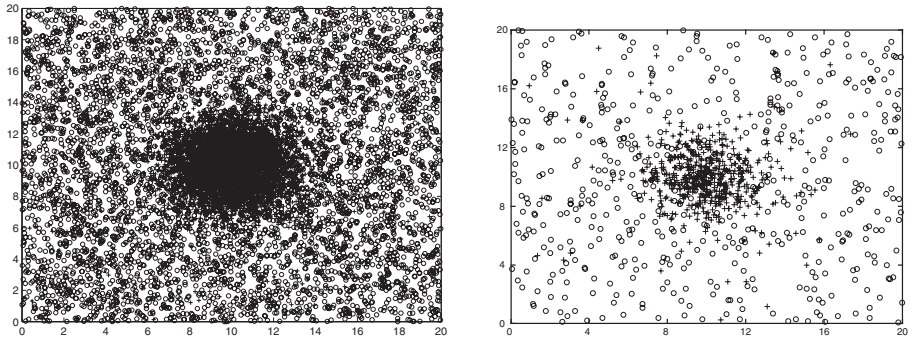
Figure 3.21. Example of data set that cannot be partitioned optimally using a decision tree with single attribute test conditions. The true decision boundary is shown by the dashed line.

Although an oblique decision tree is more expressive and can produce more compact trees, finding the optimal test condition is computationally more expensive.

9. **Choice of Impurity Measure:** It should be noted that the choice of impurity measure often has little effect on the performance of decision tree classifiers since many of the impurity measures are quite consistent with each other, as shown in Figure 3.11 on page 149. Instead, the strategy used to prune the tree has a greater impact on the final tree than the choice of impurity measure.

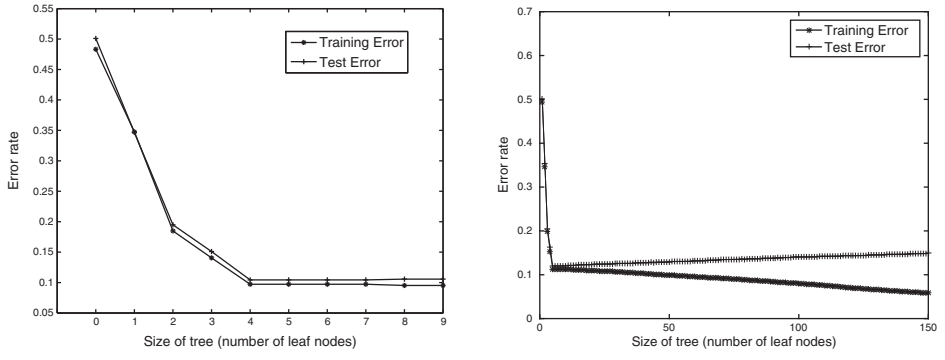
3.4 Model Overfitting

Methods presented so far try to learn classification models that show the lowest error on the training set. However, as we will show in the following example, even if a model fits well over the training data, it can still show poor generalization performance, a phenomenon known as model overfitting.



(a) Example of a 2-D data. (b) Training set using 10% data.

Figure 3.22. Examples of training and test sets of a two-dimensional classification problem.



(a) Varying tree size from 1 to 8. (b) Varying tree size from 1 to 150.

Figure 3.23. Effect of varying tree size (number of leaf nodes) on training and test errors.

Example 3.5. [Overfitting and Underfitting of Decision Trees] Consider the two-dimensional data set shown in Figure 3.22(a). The data set contains instances that belong to two separate classes, represented as + and o, respectively, where each class has 5400 instances. All instances belonging to the o class were generated from a uniform distribution. For the + class, 5000 instances were generated from a Gaussian distribution centered at (10,10) with unit variance, while the remaining 400 instances were sampled from the same uniform distribution as the o class. We can see from Figure 3.22(a) that

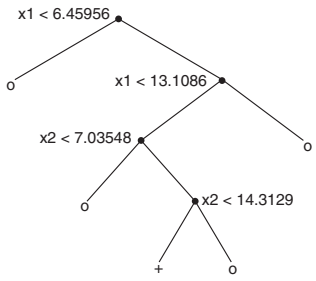
the + class can be largely distinguished from the o class by drawing a circle of appropriate size centered at (10,10). To learn a classifier using this two-dimensional data set, we randomly sampled 10% of the data for training and used the remaining 90% for testing. The training set, shown in Figure 3.22(b), looks quite representative of the overall data. We used Gini index as the impurity measure to construct decision trees of increasing sizes (number of leaf nodes), by recursively expanding a node into child nodes till every leaf node was pure, as described in Section 3.3.4.

Figure 3.23(a) shows changes in the training and test error rates as the size of the tree varies from 1 to 8. Both error rates are initially large when the tree has only one or two leaf nodes. This situation is known as **model underfitting**. Underfitting occurs when the learned decision tree is too simplistic, and thus, incapable of fully representing the true relationship between the attributes and the class labels. As we increase the tree size from 1 to 8, we can observe two effects. First, both the error rates decrease since larger trees are able to represent more complex decision boundaries. Second, the training and test error rates are quite close to each other, which indicates that the performance on the training set is fairly representative of the generalization performance. As we further increase the size of the tree from 8 to 150, the training error continues to steadily decrease till it eventually reaches zero, as shown in Figure 3.23(b). However, in a striking contrast, the test error rate ceases to decrease any further beyond a certain tree size, and then it begins to increase. The training error rate thus grossly under-estimates the test error rate once the tree becomes too large. Further, the gap between the training and test error rates keeps on widening as we increase the tree size. This behavior, which may seem counter-intuitive at first, can be attributed to the phenomena of **model overfitting**.

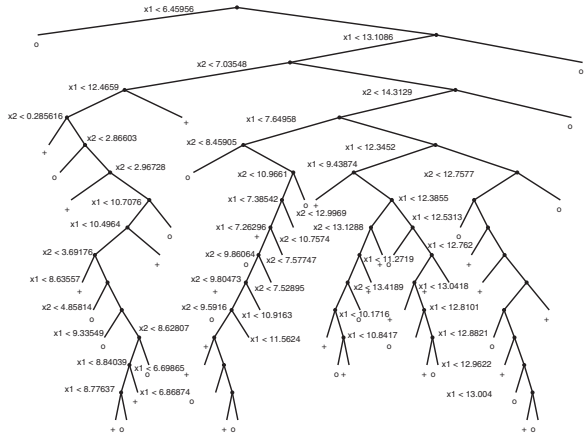
■

3.4.1 Reasons for Model Overfitting

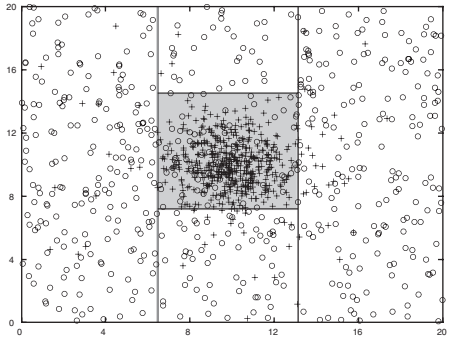
Model overfitting is the phenomena where, in the pursuit of minimizing the training error rate, an overly complex model is selected that captures specific patterns in the training data but fails to learn the *true* nature of relationships between attributes and class labels in the overall data. To illustrate this, Figure 3.24 shows decision trees and their corresponding decision boundaries (shaded rectangles represent regions assigned to the + class) for two trees of sizes 5 and 50. We can see that the decision tree of size 5 appears quite simple and its decision boundaries provide a reasonable approximation to the ideal decision boundary, which in this case corresponds to a circle centered around



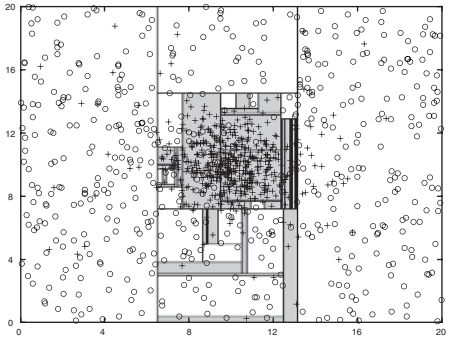
(a) Decision tree with 5 leaf nodes.



(b) Decision tree with 50 leaf nodes.



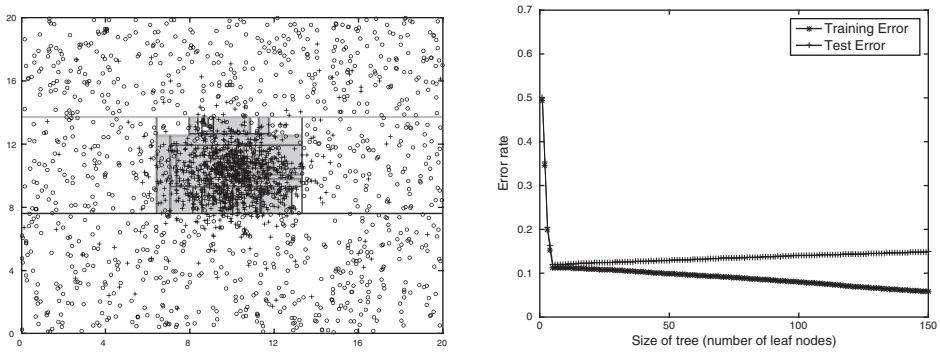
(c) Decision boundary for tree with 5 leaf nodes.



(d) Decision boundary for tree with 50 leaf nodes.

Figure 3.24. Decision trees with different model complexities.

the Gaussian distribution at (10, 10). Although its training and test error rates are non-zero, they are very close to each other, which indicates that the patterns learned in the training set should generalize well over the test set. On the other hand, the decision tree of size 50 appears much more complex than the tree of size 5, with complicated decision boundaries. For example, some of its shaded rectangles (assigned the + class) attempt to cover narrow regions in the input space that contain only one or two + training instances. Note that



(a) Decision boundary for tree with 50 leaf nodes using 20% data for training.

(b) Training and test error rates using 20% data for training.

Figure 3.25. Performance of decision trees using 20% data for training (twice the original training size).

the prevalence of + instances in such regions is highly specific to the training set, as these regions are mostly dominated by − instances in the overall data. Hence, in an attempt to perfectly fit the training data, the decision tree of size 50 starts fine tuning itself to specific patterns in the training data, leading to poor performance on an independently chosen test set.

There are a number of factors that influence model overfitting. In the following, we provide brief descriptions of two of the major factors: limited training size and high model complexity. Though they are not exhaustive, the interplay between them can help explain most of the common model overfitting phenomena in real-world applications.

Limited Training Size

Note that a training set consisting of a finite number of instances can only provide a limited representation of the overall data. Hence, it is possible that the patterns learned from a training set do not fully represent the true patterns in the overall data, leading to model overfitting. In general, as we increase the size of a training set (number of training instances), the patterns learned from the training set start resembling the true patterns in the overall data. Hence, the effect of overfitting can be reduced by increasing the training size, as illustrated in the following example.

Example 3.6. [Effect of Training Size] Suppose that we use twice the number of training instances than what we had used in the experiments conducted in Example 3.5. Specifically, we use 20% data for training and use the remainder for testing. Figure 3.25(b) shows the training and test error rates as the size of the tree is varied from 1 to 150. There are two major differences in the trends shown in this figure and those shown in Figure 3.23(b) (using only 10% of the data for training). First, even though the training error rate decreases with increasing tree size in both figures, its rate of decrease is much smaller when we use twice the training size. Second, for a given tree size, the gap between the training and test error rates is much smaller when we use twice the training size. These differences suggest that the patterns learned using 20% of data for training are more generalizable than those learned using 10% of data for training.

Figure 3.25(a) shows the decision boundaries for the tree of size 50, learned using 20% of data for training. In contrast to the tree of the same size learned using 10% data for training (see Figure 3.24(d)), we can see that the decision tree is not capturing specific patterns of noisy + instances in the training set. Instead, the high model complexity of 50 leaf nodes is being effectively used to learn the boundaries of the + instances centered at (10, 10).

■

High Model Complexity

Generally, a more complex model has a better ability to represent complex patterns in the data. For example, decision trees with larger number of leaf nodes can represent more complex decision boundaries than decision trees with fewer leaf nodes. However, an overly complex model also has a tendency to learn specific patterns in the training set that do not generalize well over unseen instances. Models with high complexity should thus be judiciously used to avoid overfitting.

One measure of model complexity is the number of "parameters" that need to be inferred from the training set. For example, in the case of decision tree induction, the attribute test conditions at internal nodes correspond to the parameters of the model that need to be inferred from the training set. A decision tree with larger number of attribute test conditions (and consequently more leaf nodes) thus involves more "parameters" and hence is more complex.

Given a class of models with a certain number of parameters, a learning algorithm attempts to select the best combination of parameter values that maximizes an evaluation metric (e.g., accuracy) over the training set. If the number of parameter value combinations (and hence the complexity) is large,

the learning algorithm has to select the best combination from a large number of possibilities, using a limited training set. In such cases, there is a high chance for the learning algorithm to pick a *spurious* combination of parameters that maximizes the evaluation metric just by random chance. This is similar to the **multiple comparisons problem** (also referred as multiple testing problem) in statistics.

As an illustration of the multiple comparisons problem, consider the task of predicting whether the stock market will rise or fall in the next ten trading days. If a stock analyst simply makes random guesses, the probability that her prediction is correct on any trading day is 0.5. However, the probability that she will predict correctly at least nine out of ten times is

$$\frac{\binom{10}{9} + \binom{10}{10}}{2^{10}} = 0.0107,$$

which is extremely low.

Suppose we are interested in choosing an investment advisor from a pool of 200 stock analysts. Our strategy is to select the analyst who makes the most number of correct predictions in the next ten trading days. The flaw in this strategy is that even if all the analysts make their predictions in a random fashion, the probability that at least one of them makes at least nine correct predictions is

$$1 - (1 - 0.0107)^{200} = 0.8847,$$

which is very high. Although each analyst has a low probability of predicting at least nine times correctly, considered together, we have a high probability of finding at least one analyst who can do so. However, there is no guarantee in the future that such an analyst will continue to make accurate predictions by random guessing.

How does the multiple comparisons problem relate to model overfitting? In the context of learning a classification model, each combination of parameter values corresponds to an analyst, while the number of training instances corresponds to the number of days. Analogous to the task of selecting the best analyst who makes the most accurate predictions on consecutive days, the task of a learning algorithm is to select the best combination of parameters that results in the highest accuracy on the training set. If the number of parameter combinations is large but the training size is small, it is highly likely for the learning algorithm to choose a spurious parameter combination that provides high training accuracy just by random chance. In the following example, we illustrate the phenomena of overfitting due to multiple comparisons in the context of decision tree induction.

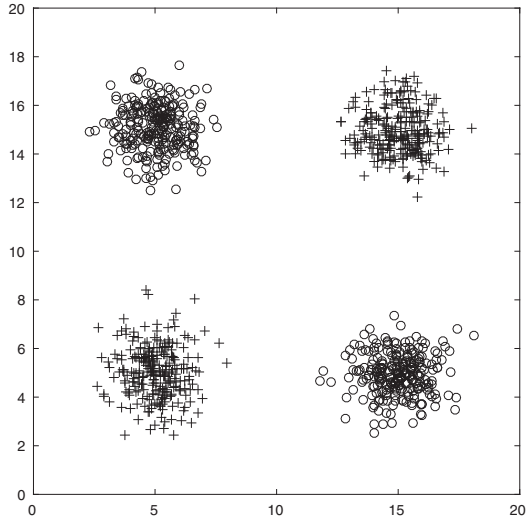
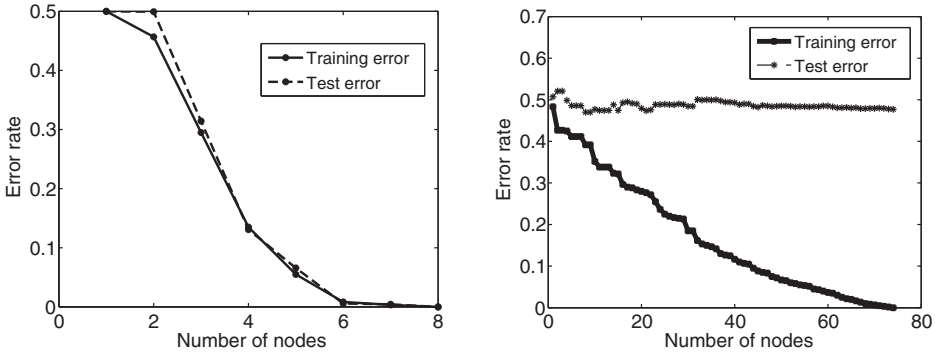


Figure 3.26. Example of a two-dimensional (X-Y) data set.

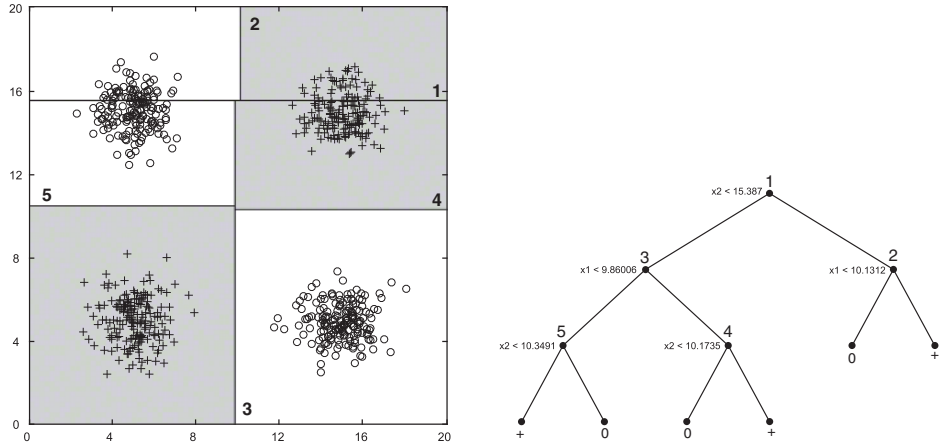


(a) Using X and Y attributes only.

(b) After adding 100 irrelevant attributes.

Figure 3.27. Training and test error rates illustrating the effect of multiple comparisons problem on model overfitting.

Example 3.7. [Multiple Comparisons and Overfitting] Consider the two-dimensional data set shown in Figure 3.26 containing 500 + and 500 o instances, which is similar to the data shown in Figure 3.19. In this data set, the distributions of both classes are well-separated in the two-dimensional (X-Y) attribute space, but none of the two attributes (X or Y) are sufficiently informative to be used alone for separating the two classes. Hence, splitting



(a) Decision boundary for tree with 6 leaf nodes.

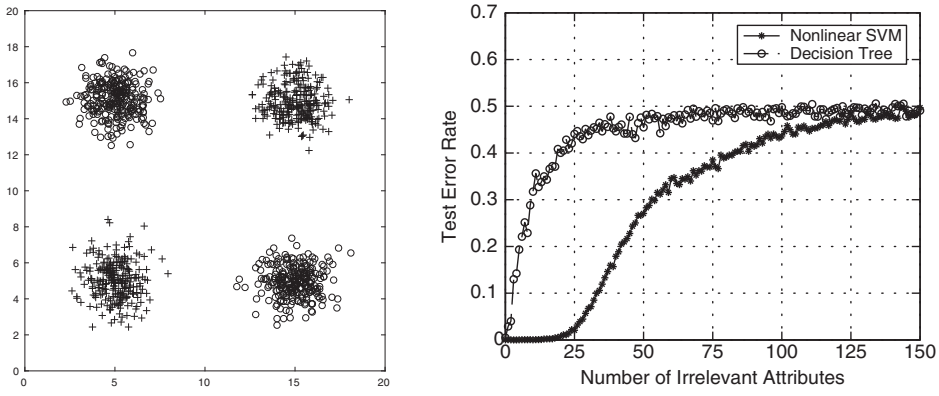
(b) Decision tree with 6 leaf nodes.

Figure 3.28. Decision tree with 6 leaf nodes using X and Y as attributes. Splits have been numbered from 1 to 5 in order of other occurrence in the tree.

the data set based on any value of an X or Y attribute will provide close to zero reduction in an impurity measure. However, if X and Y attributes are used together in the splitting criterion (e.g., splitting X at 10 *and* Y at 10), the two classes can be effectively separated.

Figure 3.27(a) shows the training and test error rates for learning decision trees of varying sizes, when 30% of the data is used for training and the remainder of the data for testing. We can see that the two classes can be separated using a small number of leaf nodes. Figure 3.28 shows the decision boundaries for the tree with six leaf nodes, where the splits have been numbered according to their order of appearance in the tree. Note that the even though splits 1 and 3 provide trivial gains, their consequent splits (2, 4, and 5) provide large gains, resulting in effective discrimination of the two classes.

Assume we add 100 irrelevant attributes to the two-dimensional X-Y data. Learning a decision tree from this resultant data will be challenging because the number of candidate attributes to choose for splitting at every internal node will increase from two to 102. With such a large number of candidate attribute test conditions to choose from, it is quite likely that spurious attribute test conditions will be selected at internal nodes because of the multiple comparisons problem. Figure 3.27(b) shows the training and test error rates after adding 100 irrelevant attributes to the training set. We can see that the



(a) Original data with two attributes.

(b) Test error rates after adding irrelevant attributes to the original data.

Figure 6.41. Comparing the effect of adding irrelevant attributes on the performance of nonlinear SVMs and decision trees.

4. SVM can be applied to categorical data by introducing dummy variables for each categorical attribute value present in the data. For example, if `Marital Status` has three values {`Single`, `Married`, `Divorced`}, we can introduce a binary variable for each of the attribute values.

5. The SVM formulation presented in this chapter is for binary class problems. However, multiclass extensions of SVM have also been proposed.

6. Although the training time of an SVM model can be large, the learned parameters can be succinctly represented with the help of a small number of support vectors, making the classification of test instances quite fast.

6.10 Ensemble Methods

This section presents techniques for improving classification accuracy by aggregating the predictions of multiple classifiers. These techniques are known as **ensemble** or **classifier combination** methods. An ensemble method constructs a set of **base classifiers** from training data and performs classification by taking a vote on the predictions made by each base classifier. This section explains why ensemble methods tend to perform better than any single classifier and presents techniques for constructing the classifier ensemble.

6.10.1 Rationale for Ensemble Method

The following example illustrates how an ensemble method can improve a classifier's performance.

Example 6.8. Consider an ensemble of 25 binary classifiers, each of which has an error rate of $\epsilon = 0.35$. The ensemble classifier predicts the class label of a test example by taking a majority vote on the predictions made by the base classifiers. If the base classifiers are identical, then all the base classifiers will commit the same mistakes. Thus, the error rate of the ensemble remains 0.35. On the other hand, if the base classifiers are independent—i.e., their errors are uncorrelated—then the ensemble makes a wrong prediction only if more than half of the base classifiers predict incorrectly. In this case, the error rate of the ensemble classifier is

$$e_{\text{ensemble}} = \sum_{i=13}^{25} \binom{25}{i} \epsilon^i (1-\epsilon)^{25-i} = 0.06, \tag{6.101}$$

which is considerably lower than the error rate of the base classifiers. ■

Figure 6.42 shows the error rate of an ensemble of 25 binary classifiers (e_{ensemble}) for different base classifier error rates (ϵ). The diagonal line represents the case in which the base classifiers are identical, while the solid line represents the case in which the base classifiers are independent. Observe that the ensemble classifier performs worse than the base classifiers when ϵ is larger than 0.5.

The preceding example illustrates two necessary conditions for an ensemble classifier to perform better than a single classifier: (1) the base classifiers should be independent of each other, and (2) the base classifiers should do better than a classifier that performs random guessing. In practice, it is difficult to ensure total independence among the base classifiers. Nevertheless, improvements in classification accuracies have been observed in ensemble methods in which the base classifiers are somewhat correlated.

6.10.2 Methods for Constructing an Ensemble Classifier

A logical view of the ensemble method is presented in Figure 6.43. The basic idea is to construct multiple classifiers from the original data and then aggregate their predictions when classifying unknown examples. The ensemble of classifiers can be constructed in many ways:

1. **By manipulating the training set.** In this approach, multiple training sets are created by resampling the original data according to some

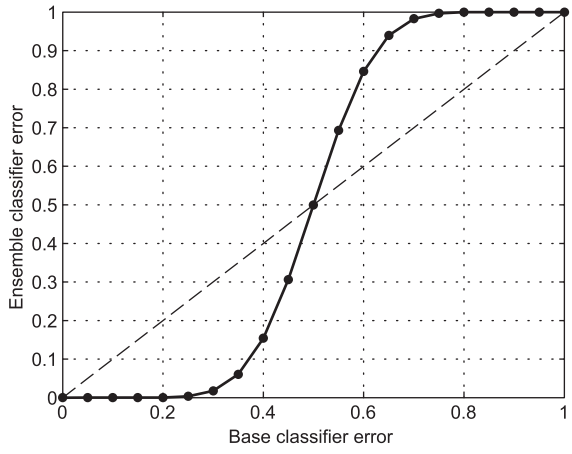


Figure 6.42. Comparison between errors of base classifiers and errors of the ensemble classifier.

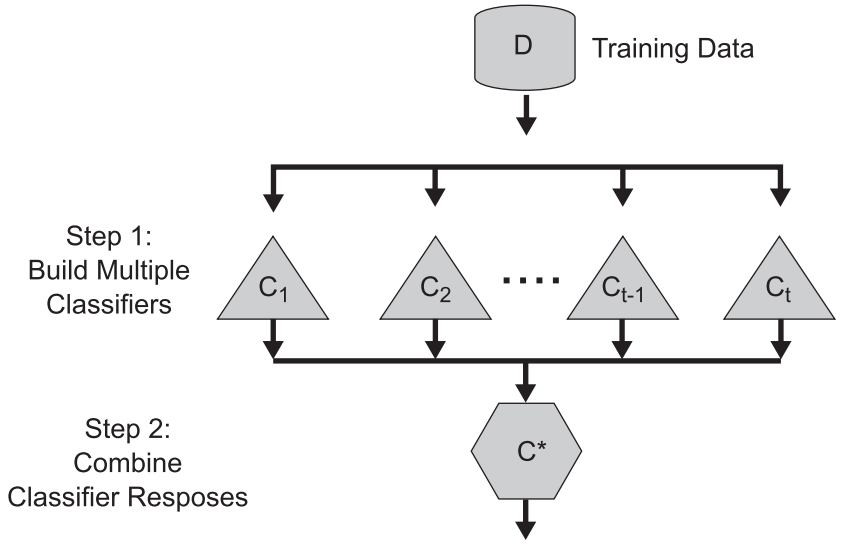


Figure 6.43. A logical view of the ensemble learning method.

sampling distribution and constructing a classifier from each training set. The sampling distribution determines how likely it is that an example will be selected for training, and it may vary from one trial to another. **Bagging** and **boosting** are two examples of ensemble methods that manipulate their training sets. These methods are described in further detail in Sections 6.10.4 and 6.10.5.

2. **By manipulating the input features.** In this approach, a subset of input features is chosen to form each training set. The subset can be either chosen randomly or based on the recommendation of domain experts. Some studies have shown that this approach works very well with data sets that contain highly redundant features. **Random forest**, which is described in Section 6.10.6, is an ensemble method that manipulates its input features and uses decision trees as its base classifiers.

3. **By manipulating the class labels.** This method can be used when the number of classes is sufficiently large. The training data is transformed into a binary class problem by randomly partitioning the class labels into two disjoint subsets, A_0 and A_1. Training examples whose class label belongs to the subset A_0 are assigned to class 0, while those that belong to the subset A_1 are assigned to class 1. The relabeled examples are then used to train a base classifier. By repeating this process multiple times, an ensemble of base classifiers is obtained. When a test example is presented, each base classifier C_i is used to predict its class label. If the test example is predicted as class 0, then all the classes that belong to A_0 will receive a vote. Conversely, if it is predicted to be class 1, then all the classes that belong to A_1 will receive a vote. The votes are tallied and the class that receives the highest vote is assigned to the test example. An example of this approach is the **error-correcting output coding** method described on page 533.

4. **By manipulating the learning algorithm.** Many learning algorithms can be manipulated in such a way that applying the algorithm several times on the same training data will result in the construction of different classifiers. For example, an artificial neural network can change its network topology or the initial weights of the links between neurons. Similarly, an ensemble of decision trees can be constructed by injecting randomness into the tree-growing procedure. For example, instead of choosing the best splitting attribute at each node, we can randomly choose one of the top k attributes for splitting.

The first three approaches are generic methods that are applicable to any classifier, whereas the fourth approach depends on the type of classifier used. The base classifiers for most of these approaches can be generated sequentially (one after another) or in parallel (all at once). Once an ensemble of classifiers has been learned, a test example $\mathbf{x}$ is classified by combining the predictions

made by the base classifiers $C_i(\mathbf{x})$:

$$C^*(\mathbf{x}) = f(C_1(\mathbf{x}), C_2(\mathbf{x}), \ldots, C_k(\mathbf{x})).$$

where f is the function that combines the ensemble responses. One simple approach for obtaining $C^*(\mathbf{x})$ is to take a majority vote of the individual predictions. An alternate approach is to take a *weighted* majority vote, where the weight of a base classifier denotes its accuracy or relevance.

Ensemble methods show the most improvement when used with **unstable classifiers**, i.e., base classifiers that are sensitive to minor perturbations in the training set, because of high model complexity. Although unstable classifiers may have a low *bias* in finding the optimal decision boundary, their predictions have a high *variance* for minor changes in the training set or model selection. This trade-off between bias and variance is discussed in detail in the next section. By aggregating the responses of multiple unstable classifiers, ensemble learning attempts to minimize their variance without worsening their bias.

6.10.3 Bias-Variance Decomposition

Bias-variance decomposition is a formal method for analyzing the generalization error of a predictive model. Although the analysis is slightly different for classification than regression, we first discuss the basic intuition of this decomposition by using an analogue of a regression problem.

Consider the illustrative task of reaching a target y by firing projectiles from a starting position $\mathbf{x}$, as shown in Figure 6.44. The target corresponds to the desired output at a test instance, while the starting position corresponds to its observed attributes. In this analogy, the projectile represents the model used for predicting the target using the observed attributes. Let $\hat{y}$ denote the

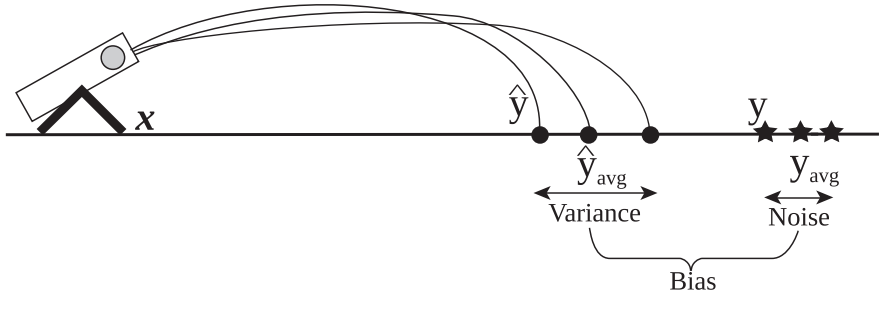


Figure 6.44. Bias-variance decomposition.

point where the projectile hits the ground, which is analogous of the prediction of the model.

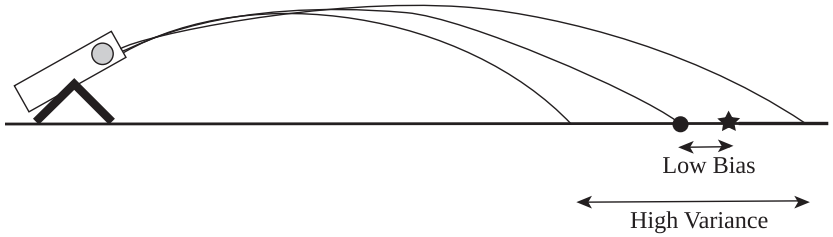
Ideally, we would like our predictions to be as close to the true target as possible. However, note that different trajectories of projectiles are possible based on differences in the training data or in the approach used for model selection. Hence, we can observe a **variance** in the predictions $\hat{y}$ over different runs of projectile. Further, the target in our example is not fixed but has some freedom to move around, resulting in a **noise** component in the true target. This can be understood as the non-deterministic nature of the output variable, where the same set of attributes can have different output values. Let $\hat{y}_{avg}$ represent the *average* prediction of the projectile over multiple runs, and y_{avg} denote the average target value. The difference between $\hat{y}_{avg}$ and y_{avg} is known as the **bias** of the model.

In the context of classification, it can be shown that the generalization error of a classification model m can be decomposed into terms involving the bias, variance, and noise components of the model in the following way:

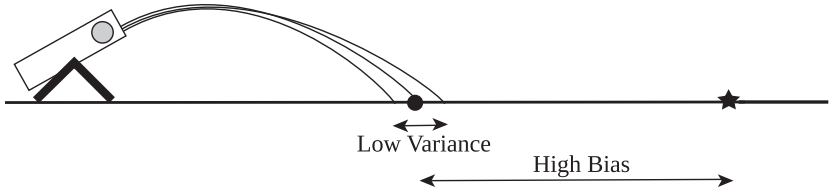
$$gen.error(m) = c_1 \times noise + bias(m) + c_2 \times variance(m)$$

where c_1 and c_2 are constants that depend on the characteristics of training and test sets. Note that while the noise term is intrinsic to the target class, the bias and variance terms depend on the choice of the classification model. The bias of a model represents how close the average prediction of the model is to the average target. Models that are able to learn complex decision boundaries, e.g., models produced by k-nearest neighbor and multi-layer ANN, generally show low bias. The variance of a model captures the *stability* of its predictions in response to minor perturbations in the training set or the model selection approach.

We can say that a model shows better generalization performance if it has a *lower bias* and *lower variance*. However, if the complexity of a model is high but the training size is small, we generally expect to see a lower bias but *higher variance*, resulting in the phenomena of overfitting. This phenomena is pictorially represented in Figure 6.45(a). On the other hand, an overly simplistic model that suffers from underfitting may show a lower variance but would suffer from a *high bias*, as shown in Figure 6.45(b). Hence, the trade-off between bias and variance provides a useful way for interpreting the effects of underfitting and overfitting on the generalization performance of a model.



(a) Phenomena of Overfitting.



(b) Phenomena of Underfitting.

Figure 6.45. Plots showing the behavior of two-dimensional solutions with constant L_2 and L_1 norms.

The bias-variance trade-off can be used to explain why ensemble learning improves the generalization performance of unstable classifiers. If a base classifier show low bias but high variance, it can become susceptible to overfitting, as even a small change in the training set will result in different predictions. However, by combining the responses of multiple base classifiers, we can expect to reduce the overall variance. Hence, ensemble learning methods show better performance primarily by lowering the variance in the predictions, although they can even help in reducing the bias. One of the simplest approaches for combining predictions and reducing their variance is to compute their average. This forms the basis of the bagging method, described in the following subsection.

6.10.4 Bagging

Bagging, which is also known as bootstrap aggregating, is a technique that repeatedly samples (with replacement) from a data set according to a uniform probability distribution. Each bootstrap sample has the same size as the original data. Because the sampling is done with replacement, some instances may appear several times in the same training set, while others may be

omitted from the training set. On average, a bootstrap sample D_i contains approximately 63% of the original training data because each sample has a probability $1 - (1 - 1/N)^N$ of being selected in each D_i. If N is sufficiently large, this probability converges to $1 - 1/e \simeq 0.632$. The basic procedure for bagging is summarized in Algorithm 6.5. After training the k classifiers, a test instance is assigned to the class that receives the highest number of votes.

To illustrate how bagging works, consider the data set shown in Table 6.4. Let x denote a one-dimensional attribute and y denote the class label. Suppose we use only one-level binary decision trees, with a test condition $x \leq k$, where k is a split point chosen to minimize the entropy of the leaf nodes. Such a tree is also known as a **decision stump**.

Table 6.4. Example of data set used to construct an ensemble of bagging classifiers.

x	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1
y	1	1	1	−1	−1	−1	−1	1	1	1

Without bagging, the best decision stump we can produce splits the instances at either $x \leq 0.35$ or $x \leq 0.75$. Either way, the accuracy of the tree is at most 70%. Suppose we apply the bagging procedure on the data set using 10 bootstrap samples. The examples chosen for training in each bagging round are shown in Figure 6.46. On the right-hand side of each table, we also describe the decision stump being used in each round.

We classify the entire data set given in Table 6.4 by taking a majority vote among the predictions made by each base classifier. The results of the predictions are shown in Figure 6.47. Since the class labels are either −1 or +1, taking the majority vote is equivalent to summing up the predicted values of y and examining the sign of the resulting sum (refer to the second to last

Algorithm 6.5 Bagging algorithm.

1: Let k be the number of bootstrap samples.
2: **for** $i = 1$ to k **do**
3: Create a bootstrap sample of size N, D_i.
4: Train a base classifier C_i on the bootstrap sample D_i.
5: **end for**
6: $C^*(x) = \underset{y}{\operatorname{argmax}} \sum_i \delta\big(C_i(x) = y\big)$.

$\{\delta(\cdot) = 1$ if its argument is true and 0 otherwise.$\}$

Bagging Round 1:

x	0.1	0.2	0.2	0.3	0.4	0.4	0.5	0.6	0.9	0.9
y	1	1	1	1	-1	-1	-1	-1	1	1

x <= 0.35 ==> y = 1
x > 0.35 ==> y = -1

Bagging Round 2:

x	0.1	0.2	0.3	0.4	0.5	0.8	0.9	1	1	1
y	1	1	1	-1	-1	1	1	1	1	1

x <= 0.65 ==> y = 1
x > 0.65 ==> y = 1

Bagging Round 3:

x	0.1	0.2	0.3	0.4	0.4	0.5	0.7	0.7	0.8	0.9
y	1	1	1	-1	-1	-1	-1	-1	1	1

x <= 0.35 ==> y = 1
x > 0.35 ==> y = -1

Bagging Round 4:

x	0.1	0.1	0.2	0.4	0.4	0.5	0.5	0.7	0.8	0.9
y	1	1	1	-1	-1	-1	-1	-1	1	1

x <= 0.3 ==> y = 1
x > 0.3 ==> y = -1

Bagging Round 5:

x	0.1	0.1	0.2	0.5	0.6	0.6	0.6	1	1	1
y	1	1	1	-1	-1	-1	-1	1	1	1

x <= 0.35 ==> y = 1
x > 0.35 ==> y = -1

Bagging Round 6:

x	0.2	0.4	0.5	0.6	0.7	0.7	0.7	0.8	0.9	1
y	1	-1	-1	-1	-1	-1	-1	1	1	1

x <= 0.75 ==> y = -1
x > 0.75 ==> y = 1

Bagging Round 7:

x	0.1	0.4	0.4	0.6	0.7	0.8	0.9	0.9	0.9	1
y	1	-1	-1	-1	-1	1	1	1	1	1

x <= 0.75 ==> y = -1
x > 0.75 ==> y = 1

Bagging Round 8:

x	0.1	0.2	0.5	0.5	0.5	0.7	0.7	0.8	0.9	1
y	1	1	-1	-1	-1	-1	-1	1	1	1

x <= 0.75 ==> y = -1
x > 0.75 ==> y = 1

Bagging Round 9:

x	0.1	0.3	0.4	0.4	0.6	0.7	0.7	0.8	1	1
y	1	1	-1	-1	-1	-1	-1	1	1	1

x <= 0.75 ==> y = -1
x > 0.75 ==> y = 1

Bagging Round 10:

x	0.1	0.1	0.1	0.1	0.3	0.3	0.8	0.8	0.9	0.9
y	1	1	1	1	1	1	1	1	1	1

x <= 0.05 ==> y = -1
x > 0.05 ==> y = 1

Figure 6.46. Example of bagging.

row in Figure 6.47). Notice that the ensemble classifier perfectly classifies all 10 examples in the original data.

The preceding example illustrates another advantage of using ensemble methods in terms of enhancing the representation of the target function. Even though each base classifier is a decision stump, combining the classifiers can lead to a decision boundary that mimics a decision tree of depth 2.

Bagging improves generalization error by reducing the variance of the base classifiers. The performance of bagging depends on the stability of the base

Round	x=0.1	x=0.2	x=0.3	x=0.4	x=0.5	x=0.6	x=0.7	x=0.8	x=0.9	x=1.0
1	1	1	1	-1	-1	-1	-1	-1	-1	-1
2	1	1	1	1	1	1	1	1	1	1
3	1	1	1	-1	-1	-1	-1	-1	-1	-1
4	1	1	1	-1	-1	-1	-1	-1	-1	-1
5	1	1	1	-1	-1	-1	-1	-1	-1	-1
6	-1	-1	-1	-1	-1	-1	-1	1	1	1
7	-1	-1	-1	-1	-1	-1	-1	1	1	1
8	-1	-1	-1	-1	-1	-1	-1	1	1	1
9	-1	-1	-1	-1	-1	-1	-1	1	1	1
10	1	1	1	1	1	1	1	1	1	1
Sum	2	2	2	-6	-6	-6	-6	2	2	2
Sign	1	1	1	-1	-1	-1	-1	1	1	1
True Class	1	1	1	-1	-1	-1	-1	1	1	1

Figure 6.47. Example of combining classifiers constructed using the bagging approach.

classifier. If a base classifier is unstable, bagging helps to reduce the errors associated with random fluctuations in the training data. If a base classifier is stable, i.e., robust to minor perturbations in the training set, then the error of the ensemble is primarily caused by bias in the base classifier. In this situation, bagging may not be able to improve the performance of the base classifiers significantly. It may even degrade the classifier's performance because the effective size of each training set is about 37% smaller than the original data.

6.10.5 Boosting

Boosting is an iterative procedure used to adaptively change the distribution of training examples for learning base classifiers so that they increasingly focus on examples that are hard to classify. Unlike bagging, boosting assigns a weight to each training example and may adaptively change the weight at the end of each boosting round. The weights assigned to the training examples can be used in the following ways:

1. They can be used to inform the sampling distribution used to draw a set of bootstrap samples from the original data.

2. They can be used to learn a model that is biased toward examples with higher weight.

This section describes an algorithm that uses weights of examples to determine the sampling distribution of its training set. Initially, the examples

are assigned equal weights, $1/N$, so that they are equally likely to be chosen for training. A sample is drawn according to the sampling distribution of the training examples to obtain a new training set. Next, a classifier is built from the training set and used to classify all the examples in the original data. The weights of the training examples are updated at the end of each boosting round. Examples that are classified incorrectly will have their weights increased, while those that are classified correctly will have their weights decreased. This forces the classifier to focus on examples that are difficult to classify in subsequent iterations.

The following table shows the examples chosen during each boosting round, when applied to the data shown in Table 6.4.

Boosting (Round 1):	7	3	2	8	7	9	4	10	6	3
Boosting (Round 2):	5	4	9	4	2	5	1	7	4	2
Boosting (Round 3):	4	4	8	10	4	5	4	6	3	4

Initially, all the examples are assigned the same weights. However, some examples may be chosen more than once, e.g., examples 3 and 7, because the sampling is done with replacement. A classifier built from the data is then used to classify all the examples. Suppose example 4 is difficult to classify. The weight for this example will be increased in future iterations as it gets misclassified repeatedly. Meanwhile, examples that were not chosen in the previous round, e.g., examples 1 and 5, also have a better chance of being selected in the next round since their predictions in the previous round were likely to be wrong. As the boosting rounds proceed, examples that are the hardest to classify tend to become even more prevalent. The final ensemble is obtained by aggregating the base classifiers obtained from each boosting round.

Over the years, several implementations of the boosting algorithm have been developed. These algorithms differ in terms of (1) how the weights of the training examples are updated at the end of each boosting round, and (2) how the predictions made by each classifier are combined. An implementation called AdaBoost is explored in the next section.

AdaBoost

Let $\{(\mathbf{x}_j, y_j) \mid j = 1, 2, \ldots, N\}$ denote a set of N training examples. In the AdaBoost algorithm, the importance of a base classifier C_i depends on its

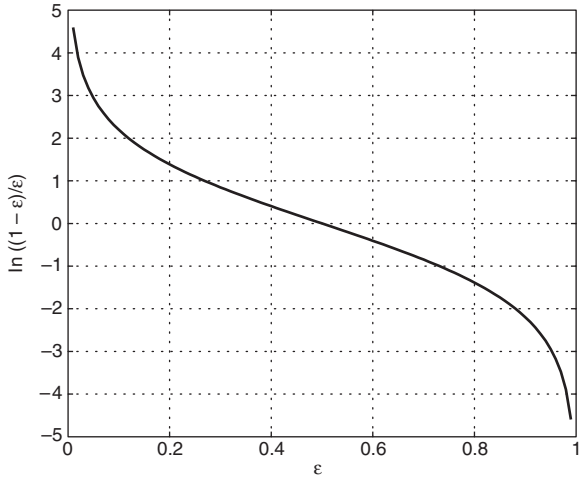


Figure 6.48. Plot of α as a function of training error ϵ.

error rate, which is defined as

$$\epsilon_i = \frac{1}{N}\left[\sum_{j=1}^{N} w_j \; I\Big(C_i(\mathbf{x}_j) \neq y_j\Big)\right], \tag{6.102}$$

where $I(p) = 1$ if the predicate p is true, and 0 otherwise. The importance of a classifier C_i is given by the following parameter,

$$\alpha_i = \frac{1}{2}\ln\left(\frac{1-\epsilon_i}{\epsilon_i}\right).$$

Note that α_i has a large positive value if the error rate is close to 0 and a large negative value if the error rate is close to 1, as shown in Figure 6.48.

The α_i parameter is also used to update the weight of the training examples. To illustrate, let $w_i^{(j)}$ denote the weight assigned to example $(\mathbf{x}_i, y_i)$ during the jth boosting round. The weight update mechanism for AdaBoost is given by the equation:

$$w_i^{(j+1)} \quad = \quad \frac{w_i^{(j)}}{Z_j} \times \begin{cases} e^{-\alpha_j} & \text{if } C_j(\mathbf{x_i}) = y_i \\ e^{\alpha_j} & \text{if } C_j(\mathbf{x_i}) \neq y_i \end{cases}, \tag{6.103}$$

where Z_j is the normalization factor used to ensure that $\sum_i w_i^{(j+1)} = 1$. The weight update formula given in Equation 6.103 increases the weights

of incorrectly classified examples and decreases the weights of those classified correctly.

Instead of using a majority voting scheme, the prediction made by each classifier C_j is weighted according to α_j. This approach allows AdaBoost to penalize models that have poor accuracy, e.g., those generated at the earlier boosting rounds. In addition, if any intermediate rounds produce an error rate higher than 50%, the weights are reverted back to their original uniform values, $w_i = 1/N$, and the resampling procedure is repeated. The AdaBoost algorithm is summarized in Algorithm 6.6.

Algorithm 6.6 AdaBoost algorithm.

1: $\mathbf{w} = \{w_j = 1/N \mid j = 1, 2, \ldots, N\}$. {Initialize the weights for all N examples.}
2: Let k be the number of boosting rounds.
3: **for** $i = 1$ to k **do**
4: Create training set D_i by sampling (with replacement) from D according to $\mathbf{w}$.
5: Train a base classifier C_i on D_i.
6: Apply C_i to all examples in the original training set, D.
7: $\epsilon_i = \frac{1}{N}\left[\sum_j w_j \, \delta\big(C_i(x_j) \neq y_j\big)\right]$ {Calculate the weighted error.}
8: **if** $\epsilon_i > 0.5$ **then**
9: $\mathbf{w} = \{w_j = 1/N \mid j = 1, 2, \ldots, N\}$. {Reset the weights for all N examples.}
10: Go back to Step 4.
11: **end if**
12: $\alpha_i = \frac{1}{2} \ln \frac{1-\epsilon_i}{\epsilon_i}$.
13: Update the weight of each example according to Equation 6.103.
14: **end for**
15: $C^*(\mathbf{x}) = \underset{y}{\operatorname{argmax}} \sum_{j=1}^{T} \alpha_j \delta(C_j(\mathbf{x}) = y)\big)$.

Let us examine how the boosting approach works on the data set shown in Table 6.4. Initially, all the examples have identical weights. After three boosting rounds, the examples chosen for training are shown in Figure 6.49(a). The weights for each example are updated at the end of each boosting round using Equation 6.103, as shown in Figure 6.50(b).

Without boosting, the accuracy of the decision stump is, at best, 70%. With AdaBoost, the results of the predictions are given in Figure 6.50(b). The final prediction of the ensemble classifier is obtained by taking a weighted average of the predictions made by each base classifier, which is shown in the last row of Figure 6.50(b). Notice that AdaBoost perfectly classifies all the examples in the training data.

Boosting Round 1:

x	0.1	0.4	0.5	0.6	0.6	0.7	0.7	0.7	0.8	1
y	1	-1	-1	-1	-1	-1	-1	-1	1	1

Boosting Round 2:

x	0.1	0.1	0.2	0.2	0.2	0.2	0.3	0.3	0.3	0.3
y	1	1	1	1	1	1	1	1	1	1

Boosting Round 3:

x	0.2	0.2	0.4	0.4	0.4	0.4	0.5	0.6	0.6	0.7
y	1	1	-1	-1	-1	-1	-1	-1	-1	-1

(a) Training records chosen during boosting.

Round	x=0.1	x=0.2	x=0.3	x=0.4	x=0.5	x=0.6	x=0.7	x=0.8	x=0.9	x=1.0
1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1
2	0.311	0.311	0.311	0.01	0.01	0.01	0.01	0.01	0.01	0.01
3	0.029	0.029	0.029	0.228	0.228	0.228	0.228	0.009	0.009	0.009

(b) Weights of training records.

Figure 6.49. Example of boosting.

An important analytical result of boosting shows that the training error of the ensemble is bounded by the following expression:

$$e_{\text{ensemble}} \leq \prod_i \left[\sqrt{\epsilon_i(1-\epsilon_i)} \right], \tag{6.104}$$

where ϵ_i is the error rate of each base classifier i. If the error rate of the base classifier is less than 50%, we can write $\epsilon_i = 0.5 - \gamma_i$, where γ_i measures how much better the classifier is than random guessing. The bound on the training error of the ensemble becomes

$$e_{\text{ensemble}} \leq \prod_i \sqrt{1-4\gamma_i^2} \leq \exp\left(-2 \sum_i \gamma_i^2 \right). \tag{6.105}$$

Hence, the training error of the ensemble decreases exponentially, which leads to the fast convergence of the algorithm. By focusing on examples that are difficult to classify by base classifiers, it is able to reduce the bias of the final predictions along with the variance. AdaBoost has been shown to provide significant improvements in performance over base classifiers on a range of data

Round	Split Point	Left Class	Right Class	α
1	0.75	-1	1	1.738
2	0.05	1	1	2.7784
3	0.3	1	-1	4.1195

(a)

Round	x=0.1	x=0.2	x=0.3	x=0.4	x=0.5	x=0.6	x=0.7	x=0.8	x=0.9	x=1.0
1	-1	-1	-1	-1	-1	-1	-1	1	1	1
2	1	1	1	1	1	1	1	1	1	1
3	1	1	1	-1	-1	-1	-1	-1	-1	-1
Sum	5.16	5.16	5.16	-3.08	-3.08	-3.08	-3.08	0.397	0.397	0.397
Sign	1	1	1	-1	-1	-1	-1	1	1	1

(b)

Figure 6.50. Example of combining classifiers constructed using the AdaBoost approach.

sets. Nevertheless, because of its tendency to focus on training examples that are wrongly classified, the boosting technique can be susceptible to overfitting, resulting in poor generalization performance in some scenarios.

6.10.6 Random Forests

Random forests attempt to improve the generalization performance by constructing an ensemble of *decorrelated* decision trees. Random forests build on the idea of bagging to use a different bootstrap sample of the training data for learning decision trees. However, a key distinguishing feature of random forests from bagging is that at every internal node of a tree, the best splitting criterion is chosen among a small set of randomly selected attributes. In this way, random forests construct ensembles of decision trees by not only manipulating training instances (by using bootstrap samples similar to bagging), but also the input attributes (by using different subsets of attributes at every internal node).

Given a training set D consisting of n instances and d attributes, the basic procedure of training a random forest classifier can be summarized using the following steps:

1. Construct a bootstrap sample D_i of the training set by randomly sampling n instances (with replacement) from D.

2. Use D_i to learn a decision tree T_i as follows. At every internal node of T_i, randomly sample a set of p attributes and choose an attribute from this subset that shows the maximum reduction in an impurity measure for splitting. Repeat this procedure till every leaf is pure, i.e., containing instances from the same class.

Once an ensemble of decision trees have been constructed, their average prediction (majority vote) on a test instance is used as the final prediction of the random forest. Note that the decision trees involved in a random forest are *unpruned* trees, as they are allowed to grow to their largest possible size till every leaf is pure. Hence, the base classifiers of random forest represent unstable classifiers that have low bias but high variance, because of their large size.

Another property of the base classifiers learned in random forests is the *lack of correlation* among their model parameters and test predictions. This can be attributed to the use of an independently sampled data set D_i for learning every decision tree T_i, similar to the bagging approach. However, random forests have the additional advantage of choosing a splitting criterion at every internal node using a different (and randomly selected) subset of attributes. This property significantly helps in breaking the correlation structure, if any, among the decision trees T_i.

To realize this, consider a training set involving a large number of attributes, where only a small subset of attributes are strong predictors of the target class, whereas other attributes are weak indicators. Given such a training set, even if we consider different bootstrap samples D_i for learning T_i, we would mostly be choosing the same attributes for splitting at internal nodes, because the weak attributes would be largely overlooked when compared with the strong predictors. This can result in a considerable correlation among the trees. However, if we restrict the choice of attributes at every internal node to a random subset of attributes, we can ensure the selection of both strong and weak predictors, thus promoting diversity among the trees. This principle is utilized by random forests for creating decorrelated decision trees.

By aggregating the predictions of an ensemble of strong and decorrelated decision trees, random forests are able to reduce the variance of the trees without negatively impacting their low bias. This makes random forests quite robust to overfitting. Additionally, because of their ability to consider only a small subset of attributes at every internal node, random forests are computationally fast and robust even in high-dimensional settings.

The number of attributes to be selected at every node, p, is a hyperparameter of the random forest classifier. A small value of p can reduce the

correlation among the classifiers but may also reduce their strength. A large value can improve their strength but may result in correlated trees similar to bagging. Although common suggestions for p in the literature include $\sqrt{d}$ and $\log_2 d+1$, a suitable value of p for a given training set can always be selected by tuning it over a validation set, as described in Chapter 3. However, there is an alternative way for selecting hyper-parameters in random forests, which does not require using a separate validation set. It involves computing a reliable estimate of the generalization error rate directly during training, known as the **out-of-bag (oob)** error estimate. The oob estimate can be computed for any generic ensemble learning method that builds independent base classifiers using bootstrap samples of the training set, e.g., bagging and random forests. The approach for computing oob estimate can be described as follows.

Consider an ensemble learning method that uses an independent base classifier T_i built on a bootstrap sample of the training set D_i. Since every training instance $\mathbf{x}$ will be used for training approximately 63% of base classifiers, we can call $\mathbf{x}$ as an **out-of-bag sample** for the remaining 27% of base classifiers that did not use it for training. If we use these remaining 27% classifiers to make predictions on $\mathbf{x}$, we can obtain the **oob error** on $\mathbf{x}$ by taking their majority vote and comparing it with its class label. Note that the oob error estimates the error of 27% classifiers on an instance that was not used for training those classifiers. Hence, the oob error can be considered as a reliable estimate of generalization error. By taking the average of oob errors of all training instances, we can compute the overall oob error estimate. This can be used as an alternative to the validation error rate for selecting hyper-parameters. Hence, random forests do not need to use a separate partition of the training set for validation, as it can simultaneously train the base classifiers and compute generalization error estimates on the same data set.

Random forests have been empirically found to provide significant improvements in generalization performance that are often comparable, if not superior, to the improvements provided by the AdaBoost algorithm. Random forests are also more robust to overfitting and run much faster than the AdaBoost algorithm.

6.10.7 Empirical Comparison among Ensemble Methods

Table 6.5 shows the empirical results obtained when comparing the performance of a decision tree classifier against bagging, boosting, and random forest. The base classifiers used in each ensemble method consist of 50 decision

Table 6.6. A confusion matrix for a binary classification problem in which the classes are not equally important.

		Predicted Class	
		+	−
Actual	+	f_{++} (TP)	f_{+-} (FN)
Class	−	f_{-+} (FP)	f_{--} (TN)

6.11.2 Evaluating Performance with Class Imbalance

The most basic approach for representing a classifier's performance on a test set is to use a **confusion matrix**, as shown in Table 6.6. This table is essentially the same as Table 3.4, which was introduced in the context of evaluating classification performance in Section 3.2. A confusion matrix summarizes the number of instances predicted correctly or incorrectly by a classifier using the following four counts:

- True positive (TP) or f_{++}, which corresponds to the number of positive examples correctly predicted by the classifier.
- False positive (FP) or f_{-+} (also known as Type I error), which corresponds to the number of negative examples wrongly predicted as positive by the classifier.
- False negative (FN) or f_{+-} (also known as Type II error), which corresponds to the number of positive examples wrongly predicted as negative by the classifier.
- True negative (TN) or f_{--}, which corresponds to the number of negative examples correctly predicted by the classifier.

The confusion matrix provides a concise representation of classification performance on a given test data set. However, it is often difficult to interpret and compare the performance of classifiers using the four-dimensional representations (corresponding to the four counts) provided by their confusion matrices. Hence, the counts in the confusion matrix are often summarized using a number of **evaluation measures**. Accuracy is an example of one such measure that combines these four counts into a single value, which is used extensively when classes are balanced. However, the accuracy measure is not suitable for handling data sets with imbalanced class distributions as it tends to favor classifiers that correctly classify the majority class. In the

following, we describe other possible measures that capture different criteria of performance when working with imbalanced classes.

A basic evaluation measure is the **true positive rate** (TPR), which is defined as the fraction of positive test instances correctly predicted by the classifier:

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}}.$$

In the medical community, TPR is also known as **sensitivity**, while in the information retrieval literature, it is also called **recall** (r). A classifier with a high TPR has a high chance of correctly identifying the positive instances of the data.

Analogously to TPR, the **true negative rate** (TNR) (also known as **specificity**) is defined as the fraction of negative test instances correctly predicted by the classifier, i.e.,

$$\text{TNR} = \frac{\text{TN}}{\text{FP} + \text{TN}}.$$

A high TNR value signifies that the classifier correctly classifies any randomly chosen negative instance in the test set. A commonly used evaluation measure that is closely related to TNR is the **false positive rate** (FPR), which is defined as $1 - \text{TNR}$.

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}.$$

Similarly, we can define **false negative rate** (FNR) as $1 - \text{TPR}$.

$$\text{FNR} = \frac{\text{FN}}{\text{FN} + \text{TP}}.$$

Note that the evaluation measures defined above do not take into account the **skew** among the classes, which can be formally defined as $\alpha = P/(P+N)$, where P and N denote the number of actual positives and actual negatives, respectively. As a result, changing the relative numbers of P and N will have no effect on TPR, TNR, FPR, or FNR, since they depend only on the fraction of correct classifications for every class, independently of the other class. Furthermore, knowing the values of TPR and TNR (and consequently FNR and FPR) does not by itself help us uniquely determine all four entries of the confusion matrix. However, together with information about the skew factor, α, and the total number of instances, N, we can compute the entire confusion matrix using TPR and TNR, as shown in Table 6.7.

Table 6.7. Entries of the confusion matrix in terms of the TPR, TNR, skew, α, and total number of instances, N.

	Predicted +	Predicted −	
Actual +	TPR $\times$ $\alpha \times N$	(1 - TPR) $\times$ $\alpha \times N$	$\alpha \times N$
Actual −	(1 - TNR) $\times$ $(1-\alpha) \times N$	TNR $\times$ $(1-\alpha) \times N$	$(1-\alpha) \times N$
			N

An evaluation measure that is sensitive to the skew is **precision**, which can be defined as the fraction of correct predictions of the positive class over the total number of positive predictions, i.e.,

$$\text{Precision, } p = \frac{\text{TP}}{\text{TP} + \text{FP}}.$$

Precision is also referred as the **positive predicted value** (PPV). A classifier that has a high precision is likely to have most of its positive predictions correct. Precision is a useful measure for highly skewed test sets where the positive predictions, even though small in numbers, are required to be mostly correct. A measure that is closely related to precision is the **false discovery rate** (FDR), which can be defined as $1 - p$.

$$\text{FDR} = \frac{\text{FP}}{\text{TP} + \text{FP}}.$$

Although both FDR and FPR focus on FP, they are designed to capture different evaluation objectives and thus can take quite contrasting values, especially in the presence of class imbalance. To illustrate this, consider a classifier with the following confusion matrix.

		Predicted Class	
		+	−
Actual	+	100	0
Class	−	100	900

Since half of the positive predictions made by the classifier are incorrect, it has a FDR value of $100/(100 + 100) = 0.5$. However, its FPR is equal to $100/(100 + 900) = 0.1$, which is quite low. This example shows that in the presence of high skew (i.e., very small value of α), even a small FPR can result in high FDR. See Section 10.6 for further discussion of this issue.

Note that the evaluation measures defined above provide an incomplete representation of performance, because they either only capture the effect of

false positives (e.g., FPR and precision) or the effect of false negatives (e.g., TPR or recall), but not both. Hence, if we optimize only one of these evaluation measures, we may end up with a classifier that shows low FN but high FP, or vice-versa. For example, a classifier that declares every instance to be positive will have a perfect recall, but high FPR and very poor precision. On the other hand, a classifier that is very conservative in classifying an instance as positive (to reduce FP) may end up having high precision but very poor recall. We thus need evaluation measures that account for both types of misclassifications, FP and FN. Some examples of such evaluation measures are summarized by the following definitions.

$$\begin{aligned} \text{Positive Likelihood Ratio} &= \frac{\text{TPR}}{\text{FPR}}. \\ F_1 \text{ measure} &= \frac{2rp}{r+p} = \frac{2 \times \text{TP}}{2 \times \text{TP} + \text{FP} + \text{FN}}. \\ G \text{ measure} &= \sqrt{rp} = \frac{\text{TP}}{\sqrt{(\text{TP}+\text{FP})(\text{TP}+\text{FN})}}. \end{aligned}$$

While some of these evaluation measures are invariant to the skew (e.g., the positive likelihood ratio), others (e.g., precision and the F_1 measure) are sensitive to skew. Further, different evaluation measures capture the effects of different types of misclassification errors in various ways. For example, the F_1 measure represents a harmonic mean between recall and precision, i.e.,

$$F_1 = \frac{2}{\frac{1}{r} + \frac{1}{p}}.$$

Because the harmonic mean of two numbers tends to be closer to the smaller of the two numbers, a high value of F_1-measure ensures that both precision and recall are reasonably high. Similarly, the G measure represents the geometric mean between recall and precision. A comparison among harmonic, geometric, and arithmetic means is given in the next example.

Example 6.9. Consider two positive numbers $a = 1$ and $b = 5$. Their arithmetic mean is $\mu_a = (a + b)/2 = 3$ and their geometric mean is $\mu_g = \sqrt{ab} = 2.236$. Their harmonic mean is $\mu_h = (2 \times 1 \times 5)/6 = 1.667$, which is closer to the smaller value between a and b than the arithmetic and geometric means. ■

A generic extension of the F_1 measure is the F_β measure, which can be defined as follows.

$$F_\beta = \frac{(\beta^2+1)rp}{r+\beta^2 p} = \frac{(\beta^2+1)\times \text{TP}}{(\beta^2+1)\text{TP}+\beta^2\text{FP}+\text{FN}}. \tag{6.106}$$

Both precision and recall can be viewed as special cases of F_β by setting $\beta = 0$ and $\beta = \infty$, respectively. Low values of β make F_β closer to precision, and high values make it closer to recall.

A more general measure that captures F_β as well as accuracy is the weighted accuracy measure, which is defined by the following equation:

$$\text{Weighted accuracy} = \frac{w_1\text{TP}+w_4 TN}{w_1\text{TP}+w_2\text{FP}+w_3\text{FN}+w_4\text{TN}}. \tag{6.107}$$

The relationship between weighted accuracy and other performance measures is summarized in the following table:

Measure	w_1	w_2	w_3	w_4
Recall	1	1	0	0
Precision	1	0	1	0
F_β	β^2+1	β^2	1	0
Accuracy	1	1	1	1

6.11.3 Finding an Optimal Score Threshold

Given a suitably chosen evaluation measure E and a distribution of classification scores, $s(\mathbf{x})$, on a validation set, we can obtain the optimal score threshold s^* on the validation set using the following approach:

1. Sort the scores in increasing order of their values.
2. For every unique value of score, s, consider the classification model that assigns an instance $\mathbf{x}$ as positive only if $s(\mathbf{x}) > s$. Let $E(s)$ denote the performance of this model on the validation set.
3. Find s^* that maximizes the evaluation measure, $E(s)$.

$$s^* = \text{argmax}_s\ E(s).$$

Note that s^* can be treated as a hyper-parameter of the classification algorithm that is learned during model selection. Using s^*, we can assign a

positive label to a future test instance $\mathbf{x}$ only if $s(\mathbf{x}) > s^*$. If the evaluation measure E is skew invariant (e.g., Positive Likelihood Ratio), then we can select s^* without knowing the skew of the test set, and the resultant classifier formed using s^* can be expected to show optimal performance on the test set (with respect to the evaluation measure E). On the other hand, if E is sensitive to the skew (e.g., precision or F_1-measure), then we need to ensure that the skew of the validation set used for selecting s^* is similar to that of the test set, so that the classifier formed using s^* shows optimal test performance with respect to E. Alternatively, given an estimate of the skew of the test data, α, we can use it along with the TPR and TNR on the validation set to estimate all entries of the confusion matrix (see Table 6.7), and thus the estimate of any evaluation measure E on the test set. The score threshold s^* selected using this estimate of E can then be expected to produce optimal test performance with respect to E. Furthermore, the methodology of selecting s^* on the validation set can help in comparing the test performance of different classification algorithms, by using the optimal values of s^* for each algorithm.

6.11.4 Aggregate Evaluation of Performance

Although the above approach helps in finding a score threshold s^* that provides optimal performance with respect to a desired evaluation measure and a certain amount of skew, α, sometimes we are interested in evaluating the performance of a classifier on a number of possible score thresholds, each corresponding to a different choice of evaluation measure and skew value. Assessing the performance of a classifier over a range of score thresholds is called **aggregate evaluation** of performance. In this style of analysis, the emphasis is not on evaluating the performance of a single classifier corresponding to the optimal score threshold, but to assess the general quality of *ranking* produced by the classification scores on the test set. In general, this helps in obtaining robust estimates of classification performance that are not sensitive to specific choices of score thresholds.

ROC Curve

One of the widely-used tools for aggregate evaluation is the **receiver operating characteristic** (ROC) curve. An ROC curve is a graphical approach for displaying the trade-off between TPR and FPR of a classifier, over varying score thresholds. In an ROC curve, the TPR is plotted along the y-axis and the FPR is shown on the x-axis. Each point along the curve corresponds to a

classification model generated by placing a threshold on the test scores produced by the classifier. The following procedure describes the generic approach for computing an ROC curve:

1. Sort the test instances in increasing order of their scores.
2. Select the lowest ranked test instance (i.e., the instance with lowest score). Assign the selected instance and those ranked above it to the positive class. This approach is equivalent to classifying all the test instances as positive class. Because all the positive examples are classified correctly and the negative examples are misclassified, TPR = FPR = 1.
3. Select the next test instance from the sorted list. Classify the selected instance and those ranked above it as positive, while those ranked below it as negative. Update the counts of TP and FP by examining the actual class label of the selected instance. If this instance belongs to the positive class, the TP count is decremented and the FP count remains the same as before. If the instance belongs to the negative class, the FP count is decremented and TP count remains the same as before.
4. Repeat Step 3 and update the TP and FP counts accordingly until the highest ranked test instance is selected. At this final threshold, TPR = FPR = 0, as all instances are labeled as negative.
5. Plot the TPR against FPR of the classifier.

Example 6.10. [Generating ROC Curve] Figure 6.52 shows an example of how to compute the TPR and FPR values for every choice of score threshold. There are five positive examples and five negative examples in the test set. The class labels of the test instances are shown in the first row of the table, while the second row corresponds to the sorted score values for each instance. The next six rows contain the counts of TP, FP, TN, and FN, along with their corresponding TPR and FPR. The table is then filled from left to right. Initially, all the instances are predicted to be positive. Thus, $TP = FP = 5$ and TPR = FPR = 1. Next, we assign the test instance with the lowest score as the negative class. Because the selected instance is actually a positive example, the TP count decreases from 5 to 4 and the FP count is the same as before. The FPR and TPR are updated accordingly. This process is repeated until we reach the end of the list, where TPR = 0 and FPR=0. The ROC curve for this example is shown in Figure 6.53.

■

Class	+	−	+	−	−	−	+	−	+	+	
	0.25	0.43	0.53	0.76	0.85	0.85	0.85	0.87	0.93	0.95	1.00
TP	5	4	4	3	3	3	3	2	2	1	0
FP	5	5	4	4	3	2	1	1	0	0	0
TN	0	0	1	1	2	3	4	4	5	5	5
FN	0	1	1	2	2	2	2	3	3	4	5
TPR	1	0.8	0.8	0.6	0.6	0.6	0.6	0.4	0.4	0.2	0
FPR	1	1	0.8	0.8	0.6	0.4	0.2	0.2	0	0	0

Figure 6.52. Computing the TPR and FPR at every score threshold.

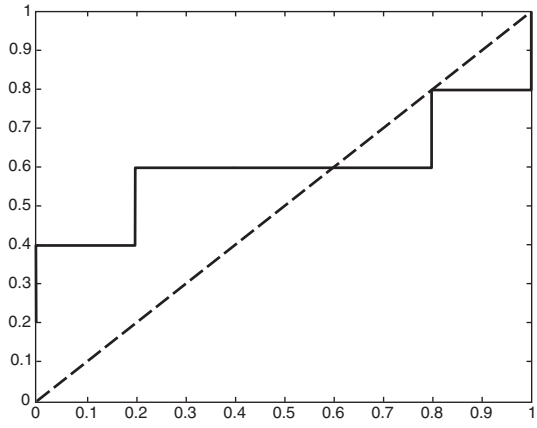


Figure 6.53. ROC curve for the data shown in Figure 6.52.

Note that in an ROC curve, the TPR monotonically increases with FPR, because the inclusion of a test instance in the set of predicted positives can either increase the TPR or the FPR. The ROC curve thus has a staircase pattern. Furthermore, there are several critical points along an ROC curve that have well-known interpretations:

(TPR=0, FPR=0): Model predicts every instance to be a negative class.
(TPR=1, FPR=1): Model predicts every instance to be a positive class.
(TPR=1, FPR=0): The *perfect* model with zero misclassifications.

A good classification model should be located as close as possible to the upper left corner of the diagram, while a model that makes random guesses should reside along the main diagonal, connecting the points ($TPR = 0, FPR = 0$) and ($TPR = 1, FPR = 1$). Random guessing means that an

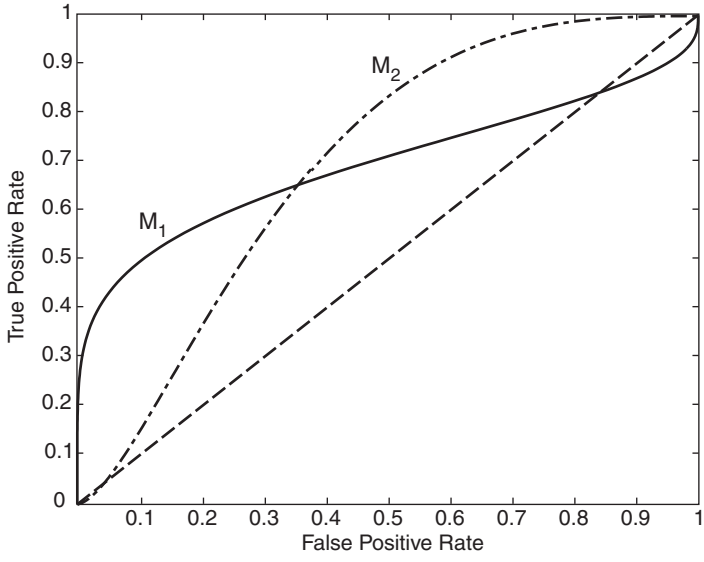


Figure 6.54. ROC curves for two different classifiers.

instance is classified as a positive class with a fixed probability p, irrespective of its attribute set. For example, consider a data set that contains n_+ positive instances and n_- negative instances. The random classifier is expected to correctly classify pn_+ of the positive instances and to misclassify pn_- of the negative instances. Therefore, the TPR of the classifier is $(pn_+)/n_+ = p$, while its FPR is $(pn_-)/p = p$. Hence, this random classifier will reside at the point (p, p) in the ROC curve along the main diagonal.

Since every point on the ROC curve represents the performance of a classifier generated using a particular score threshold, they can be viewed as different **operating points** of the classifier. One may choose one of these operating points depending on the requirements of the application. Hence, an ROC curve facilitates the comparison of classifiers over a range of operating points. For example, Figure 6.54 compares the ROC curves of two classifiers, M_1 and M_2, generated by varying the score thresholds. We can see that M_1 is better than M_2 when FPR is less than 0.36, as M_1 shows better TPR than M_2 for this range of operating points. On the other hand, M_2 is superior when FPR is greater than 0.36, since the TPR of M_2 is higher than that of M_1 for this range. Clearly, neither of the two classifiers **dominates** (is strictly better than) the other, i.e., shows higher values of TPR and lower values of FPR over all operating points.

To summarize the aggregate behavior across all operating points, one of the commonly used measures is the **area under the ROC curve** (AUC). If the classifier is perfect, then its area under the ROC curve will be equal 1. If the algorithm simply performs random guessing, then its area under the ROC curve will be equal to 0.5.

Although the AUC provides a useful summary of aggregate performance, there are certain caveats in using the AUC for comparing classifiers. First, even if the AUC of algorithm A is higher than the AUC of another algorithm B, this does not mean that algorithm A is always better than B, i.e., the ROC curve of A dominates that of B across all operating points. For example, even though M_1 shows a slightly lower AUC than M_2 in Figure 6.54, we can see that both M_1 and M_2 are useful over different ranges of operating points and none of them are strictly better than the other across all possible operating points. Hence, we cannot use the AUC to determine which algorithm is better, unless we know that the ROC curve of one of the algorithms dominates the other.

Second, although the AUC summarizes the aggregate performance over all operating points, we are often interested in only a small range of operating points in most applications. For example, even though M_1 shows slightly lower AUC than M_2, it shows higher TPR values than M_2 for small FPR values (smaller than 0.36). In the presence of class imbalance, the behavior of an algorithm over small FPR values (also termed as **early retrieval**) is often more meaningful for comparison than the performance over all FPR values. This is because, in many applications, it is important to assess the TPR achieved by a classifier in the first few instances with highest scores, without incurring a large FPR. Hence, in Figure 6.54, due to the high TPR values of M_1 during early retrieval (FPR < 0.36), we may prefer M_1 over M_2 for imbalanced test sets, despite the lower AUC of M_1. Hence, care must be taken while comparing the AUC values of different classifiers, usually by visualizing their ROC curves rather than just reporting their AUC.

A key characteristic of ROC curves is that they are agnostic to the skew in the test set, because both the evaluation measures used in constructing ROC curves (TPR and FPR) are invariant to class imbalance. Hence, ROC curves are not suitable for measuring the impact of skew on classification performance. In particular, we will obtain the same ROC curve for two test data sets that have very different skew.

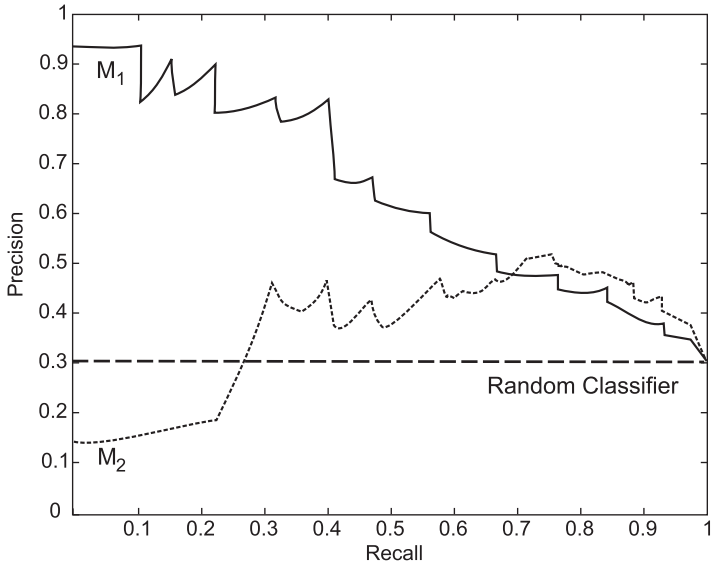


Figure 6.55. PR curves for two different classifiers.

Precision-Recall Curve

An alternate tool for aggregate evaluation is the **precision recall curve** (PR curve). The PR curve plots the precision and recall values of a classifier on the y and x axes respectively, by varying the threshold on the test scores. Figure 6.55 shows an example of PR curves for two hypothetical classifiers, M_1 and M_2. The approach for generating a PR curve is similar to the approach described above for generating an ROC curve. However, there are some key distinguishing features in the PR curve:

1. PR curves are sensitive to the skew factor $\alpha = P/(P+N)$, and different PR curves are generated for different values of α.

2. When the score threshold is lowest (every instance is labeled as positive), the precision is equal to α while recall is 1. As we increase the score threshold, the number of predicted positives can stay the same or decrease. Hence, the recall monotonically declines as the score threshold increases. In general, the precision may increase or decrease for the same value of recall, upon addition of an instance into the set of predicted positives. For example, if the kth ranked instance belongs to the negative class, then including it will result in a drop in the precision without affecting the recall. The precision may improve at the next step, which

adds the $(k+1)^{\text{th}}$ ranked instance, if this instance belongs to the positive class. Hence, the PR curve is not a smooth, monotonically increasing curve like the ROC curve, and generally has a zigzag pattern. This pattern is more prominent in the left part of the curve, where even a small change in the number of false positives can cause a large change in precision.

3. As, as we increase the imbalance among the classes (reduce the value of α), the rightmost points of all PR curves will move downwards. At and near the leftmost point on the PR curve (corresponding to larger values of score threshold), the recall is close to zero, while the precision is equal to the fraction of positives in the top ranked instances of the algorithm. Hence, different classifiers can have different values of precision at the leftmost points of the PR curve. Also, if the classification score of an algorithm monotonically varies with the posterior probability of the positive class, we can expect the PR curve to gradually decrease from a high value of precision on the leftmost point to a constant value of α at the rightmost point, albeit with some ups and downs. This can be observed in the PR curve of algorithm M_1 in Figure 6.55, which starts from a higher value of precision on the left that gradually decreases as we move towards the right. On the other hand, the PR curve of algorithm M_2 starts from a lower value of precision on the left and shows more drastic ups and downs as we move right, suggesting that the classification score of M_2 shows a weaker monotonic relationship with the posterior probability of the positive class.

4. A random classifier that assigns an instance to be positive with a fixed probability p has a precision of α and a recall of p. Hence, a classifier that performs random guessing has a horizontal PR curve with $y = \alpha$, as shown using a dashed line in Figure 6.55. Note that the random baseline in PR curves depends on the skew in the test set, in contrast to the fixed main diagonal of ROC curves that represents random classifiers.

5. Note that the precision of an algorithm is impacted more strongly by false positives in the top ranked test instances than the FPR of the algorithm. For this reason, the PR curve generally helps to magnify the differences between classifiers in the left portion of the PR curve. Hence, in the presence of class imbalance in the test data, analyzing the PR curves generally provides a deeper insight into the performance of classifiers than the ROC curves, especially in the early retrieval range of operating points.

6. The classifier corresponding to (precision = 1, recall = 1) represents the perfect classifier. Similar to AUC, we can also compute the area under the PR curve of an algorithm, known as AUC-PR. The AUC-PR of a random classifier is equal to α, while that of a perfect algorithm is equal to 1. Note that AUC-PR varies with changing skew in the test set, in contrast to the area under the ROC curve, which is insensitive to the skew. The AUC-PR helps in accentuating the differences between classification algorithms in the early retrieval range of operating points. Hence, it is more suited for evaluating classification performance in the presence of class imbalance than the area under the ROC curve. However, similar to ROC curves, a higher value of AUC-PR does not guarantee the superiority of a classification algorithm over another. This is because the PR curves of two algorithms can easily cross each other, such that they both show better performances in different ranges of operating points. Hence, it is important to visualize the PR curves before comparing their AUC-PR values, in order to ensure a meaningful evaluation.

6.12 Multiclass Problem

Some of the classification techniques described in this chapter are originally designed for binary classification problems. Yet there are many real-world problems, such as character recognition, face identification, and text classification, where the input data is divided into more than two categories. This section presents several approaches for extending the binary classifiers to handle multiclass problems. To illustrate these approaches, let $Y = \{y_1, y_2, \ldots, y_K\}$ be the set of classes of the input data.

The first approach decomposes the multiclass problem into K binary problems. For each class $y_i \in Y$, a binary problem is created where all instances that belong to y_i are considered positive examples, while the remaining instances are considered negative examples. A binary classifier is then constructed to separate instances of class y_i from the rest of the classes. This is known as the one-against-rest (1-r) approach.

The second approach, which is known as the one-against-one (1-1) approach, constructs $K(K-1)/2$ binary classifiers, where each classifier is used to distinguish between a pair of classes, (y_i, y_j). Instances that do not belong to either y_i or y_j are ignored when constructing the binary classifier for (y_i, y_j). In both 1-r and 1-1 approaches, a test instance is classified by combining the predictions made by the binary classifiers. A voting scheme is typically